KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

LESSON NOTES - SYLLABUS

RESPONSIVE WEB DESIGN & CONTENT MANAGEMENT SYSTEM PROGRAMMING

Batch:

2026

Year:

I

Semester:

I

Unit:

II

Syllabus

Unit II - Cascading Style Sheets (CSS3)

1. CSS Fundamentals
2. Selectors, Specificity, Cascade
3. Box Model and Typography
4. Colors, Backgrounds and Gradients
5. Flexbox Layout
6. CSS Grid Layout
7. Positioning and Stacking Contexts
8. Responsive Design
9. Media Queries and Container Queries
10. Pseudo-classes and Pseudo-elements

> **11. Transitions, Animations and Scroll-driven Animations**
>
> **12. Modern CSS**
>
> **13. Variables, calc(), clamp(), Nesting, @layer**

KGCAS
co-Kreate your Genius

KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

Table of Contents

1. CSS Fundamentals, Selectors, Specificity, Cascade

1.1 Topic Introduction/Overview

Cascading Style Sheets (CSS) is a stylesheet language used to describe the presentation of documents written in HTML or XML. CSS controls the layout, colors, fonts, spacing, and visual effects of web pages, separating content (HTML) from presentation (CSS). The term "cascading" refers to the priority scheme determining which style rules apply when multiple rules could affect the same element.

CSS works by selecting HTML elements and applying style rules to them. Modern CSS is a powerful language that supports animations, responsive design, custom properties, and complex layout systems. Understanding how CSS resolves conflicts between competing rules is fundamental to writing maintainable stylesheets.

The core concepts that govern how CSS applies styles include **selectors** (how to target elements), **specificity** (which rule wins when conflicts arise), and the **cascade** (the overall algorithm that determines the final applied styles).

1.2 Features, Uses & Advantages

Features:

- **Selectors**: Pattern matching to target HTML elements (element, class, ID, attribute, pseudo-class, pseudo-element selectors)
- **Specificity Calculation**: A weighted system (0,0,0,0) determining which rule applies
- **Inheritance**: Automatic passing of certain properties from parent to child elements
- **Cascade Layers (@layer)**: Modern way to explicitly control cascade order
- **Combinators**: Descendant ( ), child (>), adjacent sibling (+), and general sibling (~) selectors

- **Universal Selector (*)**: Targets every element

Uses:

- Styling text, links, and images
- Creating responsive layouts
- Animating elements
- Theming applications
- Print stylesheets and accessibility adaptations

Advantages:

- Separation of concerns (content vs. presentation)
- Consistent styling across multiple pages
- Reduced code duplication
- Better accessibility and user experience
- Easier maintenance and updates
- Smaller file sizes through reuse

1.3 Syntax & its Explanation

Basic CSS Syntax:

```
selector {
  property: value;
  property: value;
}
```

Selector Types:

```
/* Element selector */
h1 { color: blue; }

/* Class selector */
.highlight { background: yellow; }

/* ID selector */
#header { padding: 20px; }

/* Attribute selector */
input[type="text"] { border: 1px solid gray; }

/* Descendant combinator */
article p { line-height: 1.6; }

/* Child combinator */
ul > li { list-style: square; }

/* Adjacent sibling */
```

```
h2 + p { margin-top: 0; }

/* General sibling */
h2 ~ p { color: gray; }
```

Specificity Hierarchy (from lowest to highest):

1. Universal selector (*): (0,0,0,0)
2. Element/pseudo-element selectors: (0,0,0,1)
3. Class/attribute/pseudo-class selectors: (0,0,1,0)
4. ID selectors: (0,1,0,0)
5. Inline styles: (1,0,0,0)
6. `!important`: Overrides everything except another `!important`

Cascade Order (when specificity is equal):

1. Source order (last rule wins)
2. Importance (`!important`)
3. Origin (user agent < user < author)
4. Specificity

1.4 Example Programs & their Explanation

Example 1: Demonstrating Selectors and Specificity

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>CSS Selectors Demo</title>
<style>
  /* Element selector - specificity (0,0,0,1) */
  p {
    color: black;
    font-size: 16px;
  }

  /* Class selector - specificity (0,0,1,0) - WINS over element */
  .text-primary {
    color: blue;
    font-weight: bold;
  }

  /* ID selector - specificity (0,1,0,0) - WINS over class */
  #special {
    color: green;
    font-style: italic;
  }

  /* Compound selector with higher specificity */
  div.container p.highlight {
```

```
    background: yellow;
    padding: 5px;
  }

  /* Attribute selector */
  a[target="_blank"]::after {
    content: " ↗";
    color: red;
  }
</style>
</head>
<body>
  <div class="container">
    <p>This is a regular paragraph (element selector applies).</p>
    <p class="text-primary">This uses a class selector (blue, bold).</p>
    <p id="special" class="text-primary">ID wins! (green, italic).</p>
    <p class="highlight">This has highlighted background.</p>
    <a href="https://example.com" target="_blank">External link</a>
  </div>
</body>
</html>
```

Explanation: This example demonstrates how specificity determines which styles apply. The first paragraph uses the element selector's black color. The second uses the class selector's blue color and bold weight (class beats element). The third paragraph has both an ID and class, but the ID selector's green color wins because IDs have higher specificity. The highlighted paragraph shows a compound selector (`div.container p.highlight`) applying yellow background. The attribute selector adds an arrow to external links using a pseudo-element.

Example 2: Cascade and Inheritance in Action

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Cascade Demo</title>
<style>
  /* Parent sets inheritable property */
  body {
    color: #333;
    font-family: 'Segoe UI', sans-serif;
    line-height: 1.5;
  }

  /* All children inherit body color unless overridden */
  .container {
    color: #555; /* Inherited by all descendants */
  }

  /* !important beats everything except other !important */
  .urgent {
```

```
      color: red !important;
    }

    /* More specific selector wins */
    .container .urgent {
      color: orange; /* This is MORE specific but no !important */
    }

    /* Inline style would normally win, but !important in stylesheet wins */
  </style>
  </head>
  <body>
    <div class="container">
      <p>Default inherited color (#555).</p>
      <p class="urgent">This is URGENT (red with !important).</p>
      <div class="urgent">Also urgent - !important overrides specificity.</div>
    </div>
  </body>
  </html>
```

Explanation: This example shows the cascade in action. The `body` sets a base color that children inherit. The `.container` overrides this for its descendants. The `!important` declaration on `.urgent` forces the red color to win, even against the more specific `.container .urgent` selector. This demonstrates that `!important` overrides normal specificity rules, though it should be used sparingly in real projects.

1.5 Conclusion

CSS fundamentals form the bedrock of web styling. Mastering selectors allows you to precisely target elements, while understanding specificity and the cascade ensures you can predict and control which styles apply in complex scenarios. The cascade is not just a feature but a well-defined algorithm that resolves conflicts deterministically. As you advance, remember that lower specificity and clear selector strategies lead to more maintainable code. Tools like `@layer` (covered later) extend this control even further, making CSS more predictable at scale.

---

2. Box Model and Typography

2.1 Topic Introduction/Overview

The **CSS Box Model** is the foundation of layout in CSS. Every HTML element is treated as a rectangular box with four parts: **content**, **padding**, **border**, and **margin**. Understanding the box model is essential because it controls how elements take up space, how they relate to each other, and how layouts are constructed.

Typography in CSS deals with the styling of text—controlling font choices, sizes, weights, line heights, letter spacing, and more. Good typography improves readability, establishes visual hierarchy, and enhances the user experience. CSS provides extensive properties to control text presentation, from basic font-family to advanced features like variable fonts and OpenType features.

Together, the box model and typography are fundamental to creating visually appealing and well-structured web pages. They affect every element on a page, from a single character of text to complex layout components.

2.2 Features, Uses & Advantages

Box Model Features:

- **Content Area**: Where text and images appear
- **Padding**: Space between content and border (transparent)
- **Border**: A line around the padding and content
- **Margin**: Space outside the border (transparent)
- **Box-sizing**: Controls how width/height is calculated (`content-box` vs `border-box`)
- **Shorthand Properties**: `margin`, `padding`, `border` (1-4 values)

Typography Features:

- **Font Properties**: `font-family`, `font-size`, `font-weight`, `font-style`
- **Text Properties**: `text-align`, `text-transform`, `text-decoration`, `text-shadow`
- **Line Properties**: `line-height`, `letter-spacing`, `word-spacing`
- **Web Fonts**: `@font-face` and `font-display`
- **Variable Fonts**: Single file with multiple weights/styles
- **Font Loading**: `font-display`, preload strategies

Advantages:

- Predictable spacing and layout calculations
- Responsive typography with `clamp()` and viewport units
- Accessibility through proper sizing and contrast
- Brand consistency through custom fonts
- Better readability with proper line height and spacing

2.3 Syntax & its Explanation

Box Model Syntax:

```
/* Content-box (default) - width = content only */
.box {
  width: 300px;
  padding: 20px;   /* Adds to total width */
  border: 5px solid; /* Adds to total width */
  /* Total width = 300 + 40 + 10 = 350px */
}

/* Border-box - width includes padding and border */
.box {
  box-sizing: border-box;
  width: 300px;
  padding: 20px;
  border: 5px solid;
  /* Total width = 300px (content shrinks to fit) */
}

/* Shorthand for padding/margin (1, 2, 3, or 4 values) */
.element {
```

```
  margin: 10px;              /* All sides */
  margin: 10px 20px;         /* top/bottom, left/right */
  margin: 10px 20px 30px;     /* top, left/right, bottom */
  margin: 10px 20px 30px 40px; /* top, right, bottom, left (clockwise) */
}
```

Typography Syntax:

```
/* Font shorthand */
.heading {
  font: italic bold 24px/1.5 'Helvetica', sans-serif;
  /* font-style font-weight font-size/line-height font-family */
}

/* Web font with @font-face */
@font-face {
  font-family: 'CustomFont';
  src: url('font.woff2') format('woff2');
  font-display: swap;
  font-weight: 100 900; /* Variable font range */
}

/* Text styling */
.text {
  font-size: 1.125rem;
  line-height: 1.6;
  letter-spacing: 0.02em;
  word-spacing: 0.1em;
  text-transform: uppercase;
  text-decoration: underline wavy red;
  text-shadow: 1px 1px 2px rgba(0,0,0,0.3);
}
```

2.4 Example Programs & their Explanation

Example 1: Box Model Visualization

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Box Model Demo</title>
<style>
  * { box-sizing: border-box; }

  body {
    font-family: system-ui, sans-serif;
    margin: 0;
    padding: 20px;
```

```
      background: #f5f5f5;
    }

    .demo-box {
      width: 300px;
      background: #ffd6d6;     /* Content area color */
      padding: 20px;           /* Inner spacing */
      border: 8px solid #ff6b6b; /* Border */
      margin: 30px;            /* Outer spacing */
      color: #333;
    }

    .box-with-shadow {
      width: 300px;
      padding: 20px;
      background: white;
      border-radius: 8px;
      box-shadow: 0 4px 6px rgba(0,0,0,0.1);
      margin: 30px 0;
    }

    .label {
      background: rgba(0,0,0,0.7);
      color: white;
      padding: 2px 6px;
      font-size: 12px;
      border-radius: 3px;
    }
  </style>
  </head>
  <body>
    <h1>Box Model Visualization</h1>
    <div class="demo-box">
      <span class="label">Content</span> with 20px padding and 8px border.
      Margin (30px) separates it from other elements.
    </div>
    <div class="box-with-shadow">
      A modern card with border-radius and box-shadow. The box includes padding
      in its width calculation thanks to <code>box-sizing: border-box</code>.
    </div>
  </body>
  </html>
```

Explanation: This example visualizes the box model. The first box has different colors for content, padding area (visible through the background extending to the border), and margin (the surrounding empty space). The second box demonstrates a modern card UI with rounded corners and shadow. The `box-sizing: border-box` declaration makes the width include padding and border, simplifying layout calculations—a best practice used in most modern stylesheets via the `*` universal selector.

Example 2: Typography Showcase

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Typography Demo</title>
<style>
  body {
    font-family: Georgia, 'Times New Roman', serif;
    line-height: 1.6;
    color: #222;
    max-width: 700px;
    margin: 0 auto;
    padding: 20px;
  }

  h1 {
    font-family: 'Helvetica Neue', Arial, sans-serif;
    font-size: clamp(2rem, 5vw, 3.5rem);
    font-weight: 700;
    line-height: 1.1;
    letter-spacing: -0.02em;
    margin-bottom: 0.5em;
  }

  .subtitle {
    font-size: 1.25rem;
    color: #666;
    font-style: italic;
    margin-bottom: 2em;
  }

  .article-body {
    font-size: 1.125rem;
    text-align: justify;
    hyphens: auto;
  }

  .drop-cap::first-letter {
    font-size: 4em;
    float: left;
    line-height: 0.8;
    margin: 0.1em 0.1em 0 0;
    font-weight: bold;
    color: #c0392b;
  }

  .highlight {
    background: linear-gradient(180deg, transparent 60%, #ffeb3b 60%);
    padding: 0 2px;
  }
</style>
</head>
```

```
  <body>
    <article>
      <h1>The Art of Web Typography</h1>
      <p class="subtitle">Creating beautiful, readable text on the web</p>
      <div class="article-body drop-cap">
        <span class="highlight">Typography</span> is the art and technique of
        arranging type to make written language legible, readable and appealing
        when displayed. The arrangement of type involves selecting typefaces,
        point sizes, line lengths, line-spacing, and letter-spacing.
      </div>
    </article>
  </body>
  </html>
```

Explanation: This example demonstrates professional typography techniques. The `clamp()` function creates responsive font sizes that scale between mobile and desktop. The `::first-letter` pseudo-element creates a drop cap effect. The `.highlight` class uses a background gradient to create a highlighter effect that only covers 60% of the line height. The `hyphens: auto` and `text-align: justify` improve text flow. The article uses serif fonts for body text (better for long reads) and sans-serif for headings (stronger visual impact).

2.5 Conclusion

The box model and typography are inseparable aspects of CSS that affect every visual element on a page. Mastering the box model—with its content, padding, border, and margin—is essential for layout work, while proper typography makes content accessible and engaging. Using `box-sizing: border-box` is now considered a best practice, and modern typography techniques like `clamp()`, variable fonts, and advanced pseudo-elements enable responsive, beautiful text. These fundamentals underpin everything from simple pages to complex design systems.

---

3. Colors, Backgrounds, and Gradients

3.1 Topic Introduction/Overview

Colors in CSS define the visual hue, saturation, and lightness of elements. CSS supports multiple color formats including named colors, hexadecimal, RGB, HSL, and the modern `color()` function with display-p3 gamut support for wide-color displays.

Backgrounds allow you to apply colors, images, gradients, and other visual effects behind elements. The `background` shorthand combines multiple background properties for efficient styling.

Gradients are images created by CSS that smoothly transition between two or more colors. They include linear gradients (straight lines), radial gradients (from a center point), and conic gradients (rotating around a center). Gradients enable rich visual effects without image files, improving performance and scalability.

Together, these features give designers powerful tools to create vibrant, modern interfaces with depth, dimension, and visual interest.

3.2 Features, Uses & Advantages

Color Features:

- **Multiple Formats**: Named (`red`), Hex (`#ff0000`), RGB (`rgb(255,0,0)`), HSL (`hsl(0,100%,50%)`), `color(display-p3 1 0 0)`
- **Alpha Channel**: All formats support transparency (rgba, hsla, 8-digit hex)
- **`color-mix()`**: Mix two colors in any color space
- **Relative Colors**: Modify existing colors (`hsl(from var(--c) h s calc(l - 10%))`)
- **Color Spaces**: sRGB, display-p3, rec2020, prophoto-rgb, xyz

Background Features:

- **Multiple Backgrounds**: Stack images, gradients, and colors
- **Background Properties**: `background-color`, `background-image`, `background-position`, `background-size`, `background-repeat`, `background-attachment`, `background-origin`, `background-clip`
- **`background-blend-mode`**: Blend background layers together
- **`background-clip: text`**: Clip background to text shape

Gradient Features:

- **Linear Gradients**: `linear-gradient()`, `repeating-linear-gradient()`
- **Radial Gradients**: `radial-gradient()`, `repeating-radial-gradient()`
- **Conic Gradients**: `conic-gradient()`, `repeating-conic-gradient()`
- **Color Stops**: Precise control over color transitions
- **Gradient Hints**: Midpoints for smoother transitions

Advantages:

- No image files needed for gradients (faster, scalable)
- Crisp at any resolution
- Smaller file sizes
- Easy to modify and maintain
- Support for HDR and wide-gamut displays

3.3 Syntax & its Explanation

Color Syntax:

```css
.element {
  color: red;                          /* Named */
  color: #ff0000;                      /* Hex (3, 4, 6, or 8 digits) */
  color: #ff0000ff;                    /* Hex with alpha */
  color: rgb(255, 0, 0);               /* RGB */
  color: rgba(255, 0, 0, 0.5);         /* RGBA */
  color: hsl(0, 100%, 50%);            /* HSL (Hue, Saturation, Lightness) */
  color: hsla(0, 100%, 50%, 0.5);      /* HSLA */
  color: color(display-p3 1 0 0);      /* Wide gamut */
  color: transparent;                   /* Fully transparent */
  color: currentColor;                  /* Inherits from color property */
}
```

Background Syntax:

```css
/* Background shorthand */
.hero {
  background: #fff url('image.jpg') center/cover no-repeat;
  /* background-color background-image position/size repeat */
}

/* Multiple backgrounds */
.stack {
  background:
    url('top.png') top left no-repeat,
    url('middle.png') center no-repeat,
    linear-gradient(blue, red);
}
```

Gradient Syntax:

```css
/* Linear gradient - angle or direction */
.linear {
  background: linear-gradient(45deg, red, blue);
  background: linear-gradient(to right, red, blue);
  background: linear-gradient(red 0%, blue 50%, green 100%);
}

/* Radial gradient */
.radial {
  background: radial-gradient(circle at center, red, blue);
  background: radial-gradient(ellipse at top left, white, gray);
}

/* Conic gradient */
.conic {
  background: conic-gradient(red, yellow, green, blue, red);
  background: conic-gradient(from 45deg, red, blue);
```

```
}

/* Repeating gradients */
.pattern {
  background: repeating-linear-gradient(
    45deg,
    #f0f0f0,
    #f0f0f0 10px,
    #fff 10px,
    #fff 20px
  );
}
```

3.4 Example Programs & their Explanation

Example 1: Modern Gradients and Color Techniques

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Gradients Demo</title>
<style>
  body {
    margin: 0;
    font-family: system-ui, sans-serif;
    padding: 20px;
    background: #1a1a2e;
    color: white;
  }

  .gradient-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
    gap: 20px;
  }

  .card {
    aspect-ratio: 1;
    border-radius: 16px;
    padding: 20px;
    display: flex;
    align-items: flex-end;
    font-weight: bold;
    text-shadow: 0 2px 4px rgba(0,0,0,0.3);
  }

  .sunset {
    background: linear-gradient(135deg,
      #ff6b6b 0%, #feca57 50%, #ff9ff3 100%);
  }
```

```
    .mesh {
      background:
        radial-gradient(at 20% 30%, #667eea, transparent 50%),
        radial-gradient(at 80% 70%, #f093fb, transparent 50%),
        radial-gradient(at 50% 50%, #4facfe, transparent 50%),
        linear-gradient(135deg, #43e97b, #38f9d7);
    }

    .aurora {
      background: conic-gradient(from 180deg at 50% 50%,
        #00d4ff 0deg, #5b86e5 90deg, #36d1dc 180deg,
        #5b86e5 270deg, #00d4ff 360deg);
    }

    .stripes {
      background: repeating-linear-gradient(45deg,
        #ff006e, #ff006e 20px,
        #8338ec 20px, #8338ec 40px,
        #3a86ff 40px, #3a86ff 60px);
    }

    .text-fill {
      background: linear-gradient(45deg, #f093fb, #f5576c);
      -webkit-background-clip: text;
      background-clip: text;
      -webkit-text-fill-color: transparent;
      color: transparent;
      font-size: 4rem;
      font-weight: 900;
      text-align: center;
      margin-top: 30px;
    }
  </style>
  </head>
  <body>
    <h1>Modern CSS Gradients</h1>
    <div class="gradient-grid">
      <div class="card sunset">Sunset Linear</div>
      <div class="card mesh">Mesh Effect</div>
      <div class="card aurora">Aurora Conic</div>
      <div class="card stripes">Repeating Stripes</div>
    </div>
    <h2 class="text-fill">Gradient Text</h2>
  </body>
  </html>
```

Explanation: This example showcases four modern gradient techniques. The **sunset** uses a simple linear gradient with three color stops. The **mesh** effect combines multiple radial gradients with a base linear gradient to create a soft, organic look popular in modern design. The **aurora** uses a conic gradient to create rotational color flow. The **stripes** use `repeating-linear-gradient` to create a striped pattern. The "Gradient Text"

heading uses `background-clip: text` to clip a gradient to the text shape, creating colorful text without images.

Example 2: Color Manipulation and Background Blending

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Color Techniques</title>
<style>
  :root {
    --brand: #4361ee;
    --brand-light: color-mix(in srgb, var(--brand), white 30%);
    --brand-dark: color-mix(in srgb, var(--brand), black 30%);
  }

  body {
    font-family: system-ui, sans-serif;
    margin: 0;
    padding: 20px;
    background: #fafafa;
  }

  .palette {
    display: flex;
    gap: 10px;
    margin: 20px 0;
  }

  .swatch {
    width: 100px;
    height: 100px;
    border-radius: 12px;
    display: flex;
    align-items: flex-end;
    justify-content: center;
    color: white;
    padding: 8px;
    font-size: 12px;
    font-weight: bold;
  }

  .blend-demo {
    width: 100%;
    height: 300px;
    background:
      url('data:image/svg+xml;utf8,<svg xmlns="http://www.w3.org/2000/svg"
width="50" height="50"><circle cx="25" cy="25" r="20" fill="%23ff006e"/>
</svg>') center/200px no-repeat,
      linear-gradient(45deg, #ffd60a, #003566);
    background-blend-mode: multiply;
    border-radius: 12px;
```

```
      margin: 20px 0;
    }

    .glassmorphism {
      background: rgba(255, 255, 255, 0.1);
      backdrop-filter: blur(10px);
      -webkit-backdrop-filter: blur(10px);
      border: 1px solid rgba(255, 255, 255, 0.2);
      border-radius: 16px;
      padding: 30px;
      color: white;
    }

    .glass-container {
      background: linear-gradient(45deg, #667eea, #764ba2);
      padding: 40px;
      border-radius: 16px;
    }
  </style>
  </head>
  <body>
    <h1>Color Manipulation</h1>

    <h2>Brand Color Palette (using color-mix)</h2>
    <div class="palette">
      <div class="swatch" style="background: var(--brand-light)">Light</div>
      <div class="swatch" style="background: var(--brand)">Base</div>
      <div class="swatch" style="background: var(--brand-dark)">Dark</div>
    </div>

    <h2>Background Blend Mode</h2>
    <div class="blend-demo"></div>

    <h2>Glassmorphism</h2>
    <div class="glass-container">
      <div class="glassmorphism">
        <h3>Frosted Glass Effect</h3>
        <p>Using backdrop-filter for a modern glass UI effect.</p>
      </div>
    </div>
  </body>
  </html>
```

Explanation: This example demonstrates advanced color and background techniques. The `color-mix()` function creates light and dark variants of a brand color without manually calculating hex values—change the brand color and the variants update automatically. The blend demo shows how `background-blend-mode: multiply` combines an SVG image with a gradient for unique effects. The glassmorphism effect uses `backdrop-filter: blur()` to blur the background behind semi-transparent elements, creating the popular "frosted glass" UI pattern seen in macOS and modern web design.

3.5 Conclusion

Colors, backgrounds, and gradients form the visual foundation of modern web design. Modern CSS provides unprecedented control with features like `color-mix()`, wide-gamut color spaces, multiple background layers, and `background-blend-mode`. Gradients have evolved from simple linear effects to complex conic and mesh patterns, enabling rich visuals without image assets. As displays become more capable, support for HDR and wide-gamut colors will become increasingly important. These tools empower designers to create engaging, performant interfaces that scale beautifully across devices.

---

4. CSS Flexbox Layout

4.1 Topic Introduction/Overview

CSS Flexbox (Flexible Box Layout) is a one-dimensional layout system designed for distributing space and aligning items in rows or columns. It excels at solving common layout problems that were difficult with older methods like floats or table-based layouts.

Flexbox operates on the principle of a **flex container** with **flex items** as children. The container controls the overall layout direction and distribution, while items can grow, shrink, and align themselves within the available space. This makes flexbox ideal for navigation bars, card layouts, form controls, and any interface where you need flexible, content-driven sizing.

Introduced to solve real-world layout challenges, flexbox has become a cornerstone of modern CSS. It works alongside CSS Grid (which handles two-dimensional layouts) and is supported in all modern browsers.

4.2 Features, Uses & Advantages

Key Features:

- **Direction Control**: Row or column with `flex-direction`
- **Wrapping**: Items can wrap to new lines with `flex-wrap`
- **Justification**: `justify-content` distributes items along the main axis
- **Alignment**: `align-items` and `align-self` align along the cross axis
- **Flexibility**: `flex-grow`, `flex-shrink`, `flex-basis` control item sizing
- **Ordering**: `order` property changes visual order without HTML changes
- **Gaps**: `gap` property for consistent spacing between items

Common Uses:

- Navigation bars and menus
- Card layouts
- Form controls
- Centering content (vertical and horizontal)
- Sticky footers
- Equal-height columns
- Responsive components

Advantages:

- Solves vertical centering easily
- Distributes space intelligently

- Adapts to content size
- Less code than traditional methods
- Great for component-level layouts
- Handles dynamic content well

4.3 Syntax & its Explanation

Container Properties:

```
.container {
  display: flex;                    /* or inline-flex */
  flex-direction: row;              /* row | row-reverse | column | column-
reverse */
  flex-wrap: nowrap;                /* nowrap | wrap | wrap-reverse */
  flex-flow: row wrap;              /* shorthand for direction and wrap */
  justify-content: flex-start;      /* main axis distribution */
  align-items: stretch;             /* cross axis alignment */
  align-content: stretch;           /* multi-line alignment */
  gap: 20px;                        /* space between items */
}
```

Item Properties:

```
.item {
  flex: 1;                          /* grow shrink basis shorthand */
  flex-grow: 1;                     /* can grow to fill space */
  flex-shrink: 1;                   /* can shrink if needed */
  flex-basis: auto;                 /* initial size */
  align-self: center;               /* override container's align-items */
  order: 0;                         /* visual order */
}
```

Justify Content Values:

- `flex-start` (default): Items packed at start
- `flex-end`: Items packed at end
- `center`: Items centered
- `space-between`: First/last at edges, equal space between
- `space-around`: Equal space around each item
- `space-evenly`: Equal space everywhere

Align Items Values:

- `stretch` (default): Items fill cross axis
- `flex-start`, `flex-end`, `center`: Position along cross axis
- `baseline`: Align text baselines

4.4 Example Programs & their Explanation

Example 1: Navigation Bar and Card Layout

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Flexbox Layout</title>
<style>
  * { box-sizing: border-box; margin: 0; padding: 0; }

  body {
    font-family: system-ui, sans-serif;
    background: #f5f7fa;
    padding: 20px;
  }

  /* Navigation Bar */
  .navbar {
    display: flex;
    justify-content: space-between;
    align-items: center;
    background: white;
    padding: 1rem 2rem;
    border-radius: 12px;
    box-shadow: 0 2px 4px rgba(0,0,0,0.1);
    margin-bottom: 30px;
  }

  .logo {
    font-size: 1.5rem;
    font-weight: bold;
    color: #4361ee;
  }

  .nav-links {
    display: flex;
    gap: 2rem;
    list-style: none;
  }

  .nav-links a {
    text-decoration: none;
    color: #333;
    font-weight: 500;
    transition: color 0.2s;
  }

  .nav-links a:hover {
    color: #4361ee;
  }

  .cta {
    background: #4361ee;
```

```
  color: white;
  padding: 0.5rem 1.25rem;
  border-radius: 6px;
  text-decoration: none;
}

/* Card Layout */
.card-container {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
  justify-content: center;
}

.card {
  flex: 1 1 280px; /* grow shrink basis */
  max-width: 350px;
  background: white;
  border-radius: 12px;
  overflow: hidden;
  box-shadow: 0 4px 6px rgba(0,0,0,0.1);
  display: flex;
  flex-direction: column;
}

.card-image {
  height: 180px;
  background: linear-gradient(135deg, #667eea, #764ba2);
}

.card-content {
  padding: 1.5rem;
  display: flex;
  flex-direction: column;
  flex-grow: 1;
}

.card-content h3 {
  margin-bottom: 0.5rem;
}

.card-content p {
  color: #666;
  flex-grow: 1;
  margin-bottom: 1rem;
}

.card button {
  background: #4361ee;
  color: white;
  border: none;
  padding: 0.75rem;
  border-radius: 6px;
  cursor: pointer;
```

```
    }
  </style>
  </head>
  <body>
    <nav class="navbar">
      <div class="logo">Brand</div>
      <ul class="nav-links">
        <li><a href="#">Home</a></li>
        <li><a href="#">About</a></li>
        <li><a href="#">Services</a></li>
        <li><a href="#">Contact</a></li>
      </ul>
      <a href="#" class="cta">Sign Up</a>
    </nav>

    <div class="card-container">
      <div class="card">
        <div class="card-image"></div>
        <div class="card-content">
          <h3>Card Title 1</h3>
          <p>Description text that fills the available space.</p>
          <button>Learn More</button>
        </div>
      </div>
      <div class="card">
        <div class="card-image" style="background: linear-gradient(135deg,
  #f093fb, #f5576c);"></div>
        <div class="card-content">
          <h3>Card Title 2</h3>
          <p>Another card with different content length to show how flex
  maintains equal heights.</p>
          <button>Learn More</button>
        </div>
      </div>
      <div class="card">
        <div class="card-image" style="background: linear-gradient(135deg,
  #4facfe, #00f2fe);"></div>
        <div class="card-content">
          <h3>Card Title 3</h3>
          <p>Short content.</p>
          <button>Learn More</button>
        </div>
      </div>
    </div>
  </body>
  </html>
```

Explanation: This example demonstrates two common flexbox patterns. The **navbar** uses `justify-content: space-between` to push the logo left and CTA right, with `align-items: center` for vertical alignment. The **card container** uses `flex-wrap: wrap` to allow cards to wrap on smaller screens, with `flex: 1 1 280px` allowing cards to grow but never go below 280px wide. Inside each card, `display: flex; flex-`

`direction: column` with `flex-grow: 1` on the content makes cards equal height regardless of content, with the button always at the bottom. This pattern is widely used in design systems.

Example 2: Holy Grail Layout with Flexbox

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Holy Grail Layout</title>
<style>
  * { box-sizing: border-box; margin: 0; padding: 0; }

  body {
    font-family: system-ui, sans-serif;
    min-height: 100vh;
  }

  .app {
    display: flex;
    flex-direction: column;
    min-height: 100vh;
  }

  header, footer {
    background: #1a1a2e;
    color: white;
    padding: 1rem 2rem;
  }

  .main-wrapper {
    display: flex;
    flex: 1; /* Take remaining vertical space */
  }

  aside {
    background: #f5f7fa;
    padding: 1.5rem;
    flex: 0 0 200px; /* Fixed width, no grow/shrink */
  }

  main {
    flex: 1; /* Take remaining horizontal space */
    padding: 2rem;
    background: white;
  }

  aside.right {
    order: 1; /* Move to right side */
  }

  /* Responsive: stack on mobile */
  @media (max-width: 768px) {
```

```
      .main-wrapper {
        flex-direction: column;
      }

      aside {
        flex: 0 0 auto;
      }
    }
  </style>
  </head>
  <body>
    <div class="app">
      <header>Header - Fixed Height</header>

      <div class="main-wrapper">
        <aside class="left">
          <h3>Sidebar</h3>
          <p>Navigation links go here</p>
        </aside>

        <main>
          <h1>Main Content</h1>
          <p>This is the primary content area that grows to fill available space.
             The footer stays at the bottom of the viewport even with little
  content.</p>
        </main>

        <aside class="right">
          <h3>Widgets</h3>
          <p>Sidebar widgets</p>
        </aside>
      </div>

      <footer>Footer - Always at Bottom</footer>
    </div>
  </body>
  </html>
```

Explanation: This creates the classic "Holy Grail" layout: header, footer, left sidebar, main content, and right sidebar. The outer `.app` container uses `flex-direction: column` with `min-height: 100vh` to fill the viewport, and `flex: 1` on `.main-wrapper` pushes the footer down. The middle wrapper uses `display: flex` for horizontal arrangement with `aside` having fixed widths (`flex: 0 0 200px`) and `main` growing to fill space. The `order` property moves the right sidebar visually without changing HTML order. The media query stacks everything vertically on mobile—a common responsive pattern.

4.5 Conclusion

Flexbox revolutionized CSS layout by providing an intuitive, content-aware system for one-dimensional layouts. Its strength lies in distributing space along a single axis, making it perfect for component-level design like navigation bars, cards, and form controls. The combination of `flex-grow`, `flex-shrink`, and `flex-basis` gives precise control over how items respond to available space. While flexbox handles one dimension

exceptionally, two-dimensional layouts are best served by CSS Grid. Mastering both is essential for modern web development.

---

5. CSS Grid Layout

5.1 Topic Introduction/Overview

CSS Grid Layout is a two-dimensional layout system that handles both rows and columns simultaneously. Unlike flexbox (one-dimensional), grid allows you to define explicit tracks in both axes and place items precisely within them. This makes it ideal for page-level layouts, image galleries, dashboards, and any complex two-dimensional design.

Grid introduces concepts like **grid lines**, **grid tracks** (rows and columns), **grid cells**, and **grid areas**. You define the structure once, then place items using line numbers, named areas, or auto-placement. This separation of layout structure from item placement makes responsive design more predictable and maintainable.

CSS Grid works seamlessly with flexbox—use grid for the overall page structure and flexbox for component internals. This combination is the foundation of modern responsive layouts.

5.2 Features, Uses & Advantages

Key Features:

- **Grid Template**: Define rows and columns with `grid-template-rows` and `grid-template-columns`
- **Implicit Grid**: Auto-generated tracks with `grid-auto-rows` and `grid-auto-columns`
- **Named Areas**: Visual layout definition with `grid-template-areas`
- **Named Lines**: Custom names for grid lines
- **Track Sizing**: Fixed (`px`), flexible (`fr`), intrinsic (`min-content`, `max-content`, `auto`)
- `minmax()`: Define min and max track sizes
- `auto-fit` **and** `auto-fill`: Responsive track creation
- **Item Placement**: Position items by line number or name
- **Alignment**: `align-items`, `justify-items`, `place-items`, `align-self`, `justify-self`
- **Gap Properties**: `gap`, `row-gap`, `column-gap`

Common Uses:

- Page layouts (headers, sidebars, main, footers)
- Image galleries
- Dashboard interfaces
- Magazine-style layouts
- Form layouts
- Card grids that maintain alignment

Advantages:

- True two-dimensional control
- No need for wrapper divs
- Predictable, declarative layout

- Excellent for responsive design
- Works with overlapping items (z-index)
- Clean separation of structure and content

5.3 Syntax & its Explanation

Container Syntax:

```
.container {
  display: grid;                                /* or inline-grid */
  grid-template-columns: 1fr 2fr 1fr;          /* Three columns */
  grid-template-rows: 100px auto 50px;         /* Three rows */
  grid-template-areas:
    "header header header"
    "sidebar main main"
    "footer footer footer";
  gap: 20px;                                    /* row-gap column-gap */
  row-gap: 20px;
  column-gap: 10px;
}
```

Item Placement:

```
.item {
  grid-column: 1 / 3;           /* Span from line 1 to line 3 */
  grid-row: 2 / 4;
  grid-column-start: 1;
  grid-column-end: span 2;      /* Span 2 columns */
  grid-area: header;             /* Use named area */
  place-self: center;            /* Shorthand for align/justify-self */
}
```

Track Sizing Functions:

```
.grid {
  grid-template-columns:
    minmax(200px, 1fr)          /* Min 200px, max 1fr */
    minmax(100px, 300px)        /* Constrained range */
    auto;                       /* Based on content */
}
```

Responsive Grid:

```
.responsive {
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
```

```
  /* Creates as many 250px+ columns as fit, expands to fill space */
}
```

5.4 Example Programs & their Explanation

Example 1: Dashboard Layout with Named Areas

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Grid Dashboard</title>
<style>
  * { box-sizing: border-box; margin: 0; padding: 0; }

  body {
    font-family: system-ui, sans-serif;
    min-height: 100vh;
    background: #f5f7fa;
  }

  .dashboard {
    display: grid;
    grid-template-columns: 250px 1fr;
    grid-template-rows: 70px 1fr;
    grid-template-areas:
      "sidebar header"
      "sidebar main";
    min-height: 100vh;
  }

  .header {
    grid-area: header;
    background: white;
    display: flex;
    align-items: center;
    padding: 0 2rem;
    box-shadow: 0 2px 4px rgba(0,0,0,0.1);
  }

  .sidebar {
    grid-area: sidebar;
    background: #1a1a2e;
    color: white;
    padding: 2rem 1rem;
  }

  .sidebar h2 { margin-bottom: 2rem; }
  .sidebar a {
    display: block;
    color: #b8b8d4;
```

```
    text-decoration: none;
    padding: 0.75rem 1rem;
    border-radius: 6px;
    margin-bottom: 0.5rem;
  }
  .sidebar a:hover {
    background: rgba(255,255,255,0.1);
    color: white;
  }

  .main {
    grid-area: main;
    padding: 2rem;
    overflow-y: auto;
  }

  .stats-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
    gap: 1.5rem;
    margin-bottom: 2rem;
  }

  .stat-card {
    background: white;
    padding: 1.5rem;
    border-radius: 12px;
    box-shadow: 0 2px 4px rgba(0,0,0,0.05);
  }

  .stat-card h3 {
    color: #666;
    font-size: 0.875rem;
    text-transform: uppercase;
  }

  .stat-card .number {
    font-size: 2rem;
    font-weight: bold;
    color: #1a1a2e;
    margin-top: 0.5rem;
  }

  .chart-area {
    background: white;
    padding: 2rem;
    border-radius: 12px;
    min-height: 300px;
    display: grid;
    place-items: center;
    color: #999;
  }

  /* Responsive: collapse sidebar */
```

```
    @media (max-width: 768px) {
      .dashboard {
        grid-template-columns: 1fr;
        grid-template-areas:
          "header"
          "main";
      }
      .sidebar { display: none; }
    }
  </style>
  </head>
  <body>
    <div class="dashboard">
      <header class="header">
        <h1>Dashboard</h1>
      </header>

      <aside class="sidebar">
        <h2>📊 Analytics</h2>
        <a href="#">Overview</a>
        <a href="#">Reports</a>
        <a href="#">Users</a>
        <a href="#">Settings</a>
      </aside>

      <main class="main">
        <div class="stats-grid">
          <div class="stat-card">
            <h3>Total Users</h3>
            <div class="number">12,345</div>
          </div>
          <div class="stat-card">
            <h3>Revenue</h3>
            <div class="number">$45,678</div>
          </div>
          <div class="stat-card">
            <h3>Conversion</h3>
            <div class="number">3.2%</div>
          </div>
          <div class="stat-card">
            <h3>Active Now</h3>
            <div class="number">892</div>
          </div>
        </div>

        <div class="chart-area">
          Chart would render here
        </div>
      </main>
    </div>
  </body>
  </html>
```

Explanation: This example shows a complete dashboard layout. The `grid-template-areas` provides a visual representation of the layout—`"sidebar header"` means the first row has the sidebar and header side-by-side. The `grid-area` property on each section places it in the named area. The stats grid uses `auto-fit` with `minmax()` to create a responsive card grid that adapts from 4 columns to fewer as space decreases. The media query removes the sidebar on mobile and adjusts the grid template accordingly. This pattern is widely used in admin interfaces and SaaS applications.

Example 2: Image Gallery and Magazine Layout

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Magazine Layout</title>
<style>
  * { box-sizing: border-box; margin: 0; padding: 0; }

  body {
    font-family: Georgia, serif;
    background: #fafafa;
    padding: 2rem;
  }

  h1 {
    text-align: center;
    margin-bottom: 2rem;
    font-size: 2.5rem;
  }

  /* Image Gallery with auto-fit */
  .gallery {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
    gap: 1rem;
    margin-bottom: 3rem;
  }

  .gallery-item {
    aspect-ratio: 1;
    border-radius: 8px;
    overflow: hidden;
    background: linear-gradient(135deg, #667eea, #764ba2);
    display: grid;
    place-items: center;
    color: white;
    font-size: 1.5rem;
    font-weight: bold;
  }

  .gallery-item:nth-child(2) { background: linear-gradient(135deg, #f093fb,
#f5576c); }
  .gallery-item:nth-child(3) { background: linear-gradient(135deg, #4facfe,
```

```
#00f2fe); }
  .gallery-item:nth-child(4) { background: linear-gradient(135deg, #43e97b,
#38f9d7); }
  .gallery-item:nth-child(5) { background: linear-gradient(135deg, #fa709a,
#fee140); }
  .gallery-item:nth-child(6) { background: linear-gradient(135deg, #30cfd0,
#330867); }

  /* Magazine-style featured layout */
  .magazine {
    display: grid;
    grid-template-columns: repeat(6, 1fr);
    grid-template-rows: repeat(3, 200px);
    gap: 1rem;
  }

  .feature {
    background: white;
    border-radius: 8px;
    overflow: hidden;
    box-shadow: 0 2px 8px rgba(0,0,0,0.1);
    display: flex;
    flex-direction: column;
  }

  .feature-image {
    flex: 1;
    background: linear-gradient(135deg, #667eea, #764ba2);
  }

  .feature-content {
    padding: 1rem;
  }

  /* Magazine item placement - varied sizes */
  .feature-1 {
    grid-column: span 4;  /* Large feature */
    grid-row: span 2;
  }
  .feature-1 .feature-image {
    flex: 2;
  }

  .feature-2 {
    grid-column: span 2;  /* Sidebar article */
    grid-row: span 2;
  }

  .feature-3 {
    grid-column: span 2;
  }
  .feature-4 {
    grid-column: span 2;
  }
```

```
  .feature-5 {
    grid-column: span 2;
  }
</style>
</head>
<body>
  <h1>CSS Grid Gallery & Magazine</h1>

  <h2 style="margin-bottom: 1rem;">Responsive Gallery</h2>
  <div class="gallery">
    <div class="gallery-item">1</div>
    <div class="gallery-item">2</div>
    <div class="gallery-item">3</div>
    <div class="gallery-item">4</div>
    <div class="gallery-item">5</div>
    <div class="gallery-item">6</div>
  </div>

  <h2 style="margin-bottom: 1rem;">Magazine Layout</h2>
  <div class="magazine">
    <article class="feature feature-1">
      <div class="feature-image"></div>
      <div class="feature-content">
        <h3>Featured Article</h3>
        <p>Main story with large image</p>
      </div>
    </article>

    <article class="feature feature-2">
      <div class="feature-image" style="background: linear-gradient(135deg,
#f093fb, #f5576c);"></div>
      <div class="feature-content">
        <h3>Side Story</h3>
        <p>Tall sidebar article</p>
      </div>
    </article>

    <article class="feature feature-3">
      <div class="feature-image" style="background: linear-gradient(135deg,
#4facfe, #00f2fe);"></div>
      <div class="feature-content">
        <h3>Article 3</h3>
      </div>
    </article>

    <article class="feature feature-4">
      <div class="feature-image" style="background: linear-gradient(135deg,
#43e97b, #38f9d7);"></div>
      <div class="feature-content">
        <h3>Article 4</h3>
      </div>
    </article>

    <article class="feature feature-5">
```

```
      <div class="feature-image" style="background: linear-gradient(135deg,
#fa709a, #fee140);"></div>
      <div class="feature-content">
        <h3>Article 5</h3>
      </div>
    </article>
  </div>
</body>
</html>
```

Explanation: This demonstrates two advanced grid patterns. The **gallery** uses `repeat(auto-fit, minmax(200px, 1fr))` to create a responsive grid that automatically adjusts column count based on available space—no media queries needed. The **magazine layout** uses `grid-column: span N` to create varied item sizes within a 6-column grid. The featured article spans 4 columns and 2 rows, the sidebar article spans 2 columns and 2 rows, and three smaller articles each span 2 columns. This creates visual hierarchy and interest typical of magazine or news layouts. The `aspect-ratio: 1` keeps gallery items square regardless of content.

5.5 Conclusion

CSS Grid is the most powerful layout system in CSS, providing true two-dimensional control that was impossible with older techniques. Its declarative nature—where you define the structure and place items—makes complex layouts maintainable and predictable. The combination of `grid-template-areas` for visual layout definition, `auto-fit`/`minmax()` for responsive design, and item spanning for varied layouts gives unprecedented flexibility. Combined with flexbox for component internals, grid enables the full range of modern web layouts, from simple pages to complex dashboards and editorial designs.

---

6. Positioning and Stacking Contexts

6.1 Topic Introduction/Overview

CSS Positioning controls how elements are placed in the document flow. While normal flow arranges elements in their source order, positioning allows you to take elements out of flow and place them precisely using coordinates. There are five position values: `static`, `relative`, `absolute`, `fixed`, and `sticky`.

Stacking contexts determine the order in which elements are painted on the screen (their z-order). When elements overlap, the stacking context controls which appears on top. Understanding stacking contexts is crucial for complex layouts with modals, dropdowns, tooltips, and any overlapping content.

Together, positioning and stacking contexts give you precise control over visual layering and placement. They enable features like fixed navigation bars, modal dialogs, tooltips, and complex overlapping UI patterns.

6.2 Features, Uses & Advantages

Position Values:

- `static`: Default, follows normal document flow
- `relative`: Positioned relative to its normal position
- `absolute`: Positioned relative to nearest positioned ancestor

- `fixed`: Positioned relative to the viewport
- `sticky`: Hybrid—relative until scroll threshold, then fixed

Positioning Properties:

- `top`, `right`, `bottom`, `left`: Offset values
- `z-index`: Stacking order (requires positioned element)
- `inset`: Shorthand for all four offset properties

Stacking Context Triggers:

- Position with `z-index` value
- `position: fixed` or `sticky`
- Opacity less than 1
- Transform, filter, perspective properties
- `isolation: isolate`
- `will-change` with certain properties
- `mix-blend-mode` other than normal

Common Uses:

- Fixed headers and navigation
- Modal dialogs and overlays
- Tooltips and dropdowns
- Sticky table headers
- Floating action buttons
- Decorative elements

6.3 Syntax & its Explanation

Position Syntax:

```
.relative {
  position: relative;
  top: 10px;      /* Move down 10px from normal position */
  left: 20px;     /* Move right 20px from normal position */
}

.absolute {
  position: absolute;
  top: 0;
  right: 0;       /* Position in top-right of positioned ancestor */
}

.fixed {
  position: fixed;
  bottom: 20px;
  right: 20px;    /* Fixed bottom-right of viewport */
}

.sticky {
```

```
  position: sticky;
  top: 0;         /* Stick to top when scrolling */
}

.center-modal {
  position: fixed;
  inset: 0;       /* Same as top/right/bottom/left: 0 */
  display: grid;
  place-items: center;
}
```

Z-Index and Stacking:

```
.modal-backdrop {
  position: fixed;
  inset: 0;
  z-index: 1000;  /* Above normal content */
}

.dropdown {
  position: absolute;
  z-index: 100;
}

.tooltip {
  position: relative;
  z-index: 10;
}
```

Creating Stacking Contexts:

```
.isolate {
  isolation: isolate;  /* Creates new stacking context */
  /* Children's z-index won't affect outside this element */
}
```

6.4 Example Programs & their Explanation

Example 1: Modal Dialog with Backdrop

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Modal Demo</title>
<style>
  * { box-sizing: border-box; }
```

```css
body {
  font-family: system-ui, sans-serif;
  margin: 0;
  min-height: 200vh; /* Make page scrollable */
  background: #f5f7fa;
}

.content {
  max-width: 800px;
  margin: 0 auto;
  padding: 2rem;
}

/* Fixed Header */
.header {
  position: sticky;
  top: 0;
  background: white;
  padding: 1rem 2rem;
  box-shadow: 0 2px 4px rgba(0,0,0,0.1);
  z-index: 100;
}

/* Floating Action Button */
.fab {
  position: fixed;
  bottom: 30px;
  right: 30px;
  width: 60px;
  height: 60px;
  border-radius: 50%;
  background: #4361ee;
  color: white;
  border: none;
  font-size: 1.5rem;
  cursor: pointer;
  box-shadow: 0 4px 12px rgba(67, 97, 238, 0.4);
  z-index: 50;
}

/* Modal Styles */
.modal {
  position: fixed;
  inset: 0;
  background: rgba(0, 0, 0, 0.5);
  display: none;
  align-items: center;
  justify-content: center;
  z-index: 1000;
}

.modal.active {
  display: flex;
```

```
    }

    .modal-content {
      background: white;
      padding: 2rem;
      border-radius: 12px;
      max-width: 500px;
      width: 90%;
      position: relative;
      /* Creates its own stacking context */
      isolation: isolate;
    }

    .modal-close {
      position: absolute;
      top: 10px;
      right: 10px;
      background: none;
      border: none;
      font-size: 1.5rem;
      cursor: pointer;
      color: #999;
    }

    .content-block {
      background: white;
      padding: 2rem;
      margin-bottom: 1rem;
      border-radius: 8px;
      position: relative;
      /* Lower z-index to test */
    }
  </style>
  </head>
  <body>
    <header class="header">
      <h1>Positioning Demo</h1>
    </header>

    <div class="content">
      <div class="content-block">
        <h2>Scroll down to test sticky header</h2>
        <p>This is regular content. The header above uses position: sticky
           to stay at the top while scrolling.</p>
      </div>
      <div class="content-block">
        <h2>Modal Test</h2>
        <p>Click the button below to open a modal dialog.</p>
        <button
  onclick="document.getElementById('modal').classList.add('active')">
          Open Modal
        </button>
      </div>
    </div>
```

```
  <button class="fab"
onclick="document.getElementById('modal').classList.add('active')">+</button>

  <div class="modal" id="modal" onclick="if(event.target===this)
this.classList.remove('active')">
    <div class="modal-content">
      <button class="modal-close"
onclick="document.getElementById('modal').classList.remove('active')">×</button
>
      <h2>Modal Title</h2>
      <p>This modal uses position: fixed with inset: 0 to cover the entire
viewport.
         The z-index of 1000 ensures it appears above all other content.</p>
      <p>The modal content uses isolation: isolate to create its own stacking
context,
         preventing z-index issues with its children.</p>
    </div>
  </div>
</body>
</html>
```

Explanation: This example demonstrates multiple positioning techniques. The **sticky header** stays at the top when scrolling but takes up space in normal flow. The **floating action button (FAB)** uses `position: fixed` to stay in the viewport's bottom-right corner. The **modal** uses `position: fixed` with `inset: 0` to cover the entire screen, with a high `z-index` to appear above everything. The `isolation: isolate` on the modal content creates a new stacking context, preventing any internal z-index issues from affecting the modal's relationship to outside elements. The modal's backdrop uses a semi-transparent black overlay (rgba) to dim the content behind it.

Example 2: Tooltip and Dropdown Patterns

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Overlapping Elements</title>
<style>
  * { box-sizing: border-box; }

  body {
    font-family: system-ui, sans-serif;
    margin: 0;
    padding: 2rem;
    background: #f5f7fa;
  }

  /* Tooltip */
  .tooltip-container {
    position: relative;
    display: inline-block;
```

```
  margin: 50px;
}

.tooltip-trigger {
  padding: 0.75rem 1.5rem;
  background: #4361ee;
  color: white;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  font-size: 1rem;
}

.tooltip {
  position: absolute;
  bottom: 100%;
  left: 50%;
  transform: translateX(-50%) translateY(-10px);
  background: #1a1a2e;
  color: white;
  padding: 0.5rem 1rem;
  border-radius: 6px;
  white-space: nowrap;
  font-size: 0.875rem;
  opacity: 0;
  pointer-events: none;
  transition: opacity 0.2s, transform 0.2s;
  z-index: 10;
}

.tooltip::after {
  content: '';
  position: absolute;
  top: 100%;
  left: 50%;
  transform: translateX(-50%);
  border: 6px solid transparent;
  border-top-color: #1a1a2e;
}

.tooltip-container:hover .tooltip {
  opacity: 1;
  transform: translateX(-50%) translateY(-15px);
}

/* Dropdown */
.dropdown {
  position: relative;
  display: inline-block;
  margin: 50px;
}

.dropdown-toggle {
  padding: 0.75rem 1.5rem;
```

```
  background: white;
  border: 2px solid #4361ee;
  color: #4361ee;
  border-radius: 6px;
  cursor: pointer;
  font-size: 1rem;
}

.dropdown-menu {
  position: absolute;
  top: 100%;
  left: 0;
  min-width: 200px;
  background: white;
  border-radius: 8px;
  box-shadow: 0 10px 25px rgba(0,0,0,0.15);
  margin-top: 8px;
  opacity: 0;
  visibility: hidden;
  transform: translateY(-10px);
  transition: all 0.2s;
  z-index: 100;
}

.dropdown.active .dropdown-menu {
  opacity: 1;
  visibility: visible;
  transform: translateY(0);
}

.dropdown-item {
  padding: 0.75rem 1rem;
  cursor: pointer;
  border-bottom: 1px solid #f0f0f0;
}

.dropdown-item:last-child {
  border-bottom: none;
  border-radius: 0 0 8px 8px;
}

.dropdown-item:first-child {
  border-radius: 8px 8px 0 0;
}

.dropdown-item:hover {
  background: #f5f7fa;
}

/* Stacking Context Demo */
.card {
  position: relative;
  width: 300px;
  height: 200px;
```

```
      background: white;
      border-radius: 8px;
      margin: 20px;
      box-shadow: 0 2px 8px rgba(0,0,0,0.1);
      z-index: 1;
    }

    .badge {
      position: absolute;
      top: -10px;
      right: -10px;
      background: red;
      color: white;
      width: 30px;
      height: 30px;
      border-radius: 50%;
      display: grid;
      place-items: center;
      font-size: 0.75rem;
      font-weight: bold;
      z-index: 2;
    }
  </style>
  </head>
  <body>
    <h1>Overlapping UI Patterns</h1>

    <div class="tooltip-container">
      <button class="tooltip-trigger">Hover Me</button>
      <div class="tooltip">This is a tooltip!</div>
    </div>

    <div class="dropdown" id="dropdown">
      <button class="dropdown-toggle"
  onclick="this.parentElement.classList.toggle('active')">
        Dropdown ▼
      </button>
      <div class="dropdown-menu">
        <div class="dropdown-item">Profile</div>
        <div class="dropdown-item">Settings</div>
        <div class="dropdown-item">Help</div>
        <div class="dropdown-item">Logout</div>
      </div>
    </div>

    <h2>Stacking Context with Badges</h2>
    <div style="display: flex; gap: 20px;">
      <div class="card">
        <div class="badge">3</div>
        <h3 style="padding: 1rem;">Card with Badge</h3>
      </div>
      <div class="card" style="background: #4361ee; color: white;">
        <div class="badge" style="background: yellow; color: black;">5</div>
        <h3 style="padding: 1rem;">Another Card</h3>
```

```
      </div>
    </div>
  </body>
  </html>
```

Explanation: This example shows three common overlapping UI patterns. The **tooltip** is positioned absolutely relative to its container (`position: relative`), centered using `transform: translateX(-50%)`, and uses a `::after` pseudo-element to create the arrow. The **dropdown menu** is positioned absolutely below its trigger, with z-index ensuring it appears above other content. The **badge** uses absolute positioning within a relatively-positioned card to sit in the top-right corner, demonstrating how z-index controls layering within a stacking context. Note how the badge with `z-index: 2` appears above the card content, but only within the card's own stacking context.

6.5 Conclusion

Positioning and stacking contexts are essential for creating layered, interactive interfaces. The five position values serve different purposes: `static` for normal flow, `relative` as a positioning context anchor, `absolute` for precise placement within containers, `fixed` for viewport-relative elements, and `sticky` for scroll-aware behavior. Stacking contexts, while sometimes confusing, provide isolation that prevents z-index pollution across the document. Understanding that `z-index` only competes within the same stacking context is key to debugging z-index issues. Modern CSS often reduces the need for explicit positioning through flexbox and grid, but positioning remains crucial for modals, tooltips, and floating UI elements.

---

7. Responsive Design & Media Queries and Container Queries

7.1 Topic Introduction/Overview

Responsive Design is an approach to web design that makes pages render well on a variety of devices and screen sizes. It combines flexible grids, flexible images, and CSS media queries to create layouts that adapt to the viewing environment. The goal is to provide an optimal viewing experience—easy reading and navigation with minimal resizing, panning, and scrolling—across devices from mobile phones to desktop monitors.

Media Queries are CSS techniques that apply styles based on device characteristics like viewport width, height, orientation, and resolution. They enable conditional CSS that responds to the user's device.

Container Queries (a newer feature) allow elements to be styled based on the size of their *parent container* rather than the viewport. This enables truly component-based responsive design where components adapt to wherever they're placed.

Together, these technologies enable the modern responsive web, moving from page-level responsive design to component-level responsive design.

7.2 Features, Uses & Advantages

Responsive Design Features:

- **Fluid Layouts**: Using percentages, `fr` units, and viewport units
- **Flexible Images**: `max-width: 100%` and `object-fit`

- **Mobile-First Approach**: Base styles for mobile, enhance for larger screens
- **Breakpoint Strategy**: Strategic points where layout changes

Media Query Features:

- **Width/Height**: `min-width`, `max-width`, `min-height`, `max-height`
- **Orientation**: `portrait` vs `landscape`
- **Resolution**: `min-resolution` for high-DPI displays
- **Hover Capability**: `(hover: hover)` for touch vs mouse
- **Pointer Type**: `(pointer: coarse)` for touch devices
- **Color Scheme**: `prefers-color-scheme: dark`
- **Reduced Motion**: `prefers-reduced-motion: reduce`
- **Logical Operators**: `and`, `or` (comma), `not`

Container Query Features:

- **Container Types**: `inline-size`, `size`, `normal`
- `@container` **Rule**: Query parent container dimensions
- **Container Names**: Target specific containers
- `container-type`: Establishes a containment context
- `cqi`, `cqw`, `cqh` **units**: Container query units

Advantages:

- Single codebase for all devices
- Better user experience
- SEO benefits (mobile-friendly ranking)
- Future-proof designs
- Component reusability (with container queries)

7.3 Syntax & its Explanation

Media Query Syntax:

```
/* Basic media query */
@media (max-width: 768px) {
  .sidebar { display: none; }
}

/* Multiple conditions */
@media (min-width: 768px) and (max-width: 1024px) {
  .container { padding: 1rem; }
}

/* Orientation and features */
@media (orientation: landscape) and (hover: hover) {
  .tooltip { display: block; }
}

/* Modern range syntax */
@media (width >= 768px) {
```

```
  /* Equivalent to min-width: 768px */
}

@media (768px <= width <= 1024px) {
  /* Range between 768px and 1024px */
}

/* User preferences */
@media (prefers-color-scheme: dark) {
  :root {
    --bg: #1a1a1a;
    --text: #f0f0f0;
  }
}

@media (prefers-reduced-motion: reduce) {
  * {
    animation: none !important;
    transition: none !important;
  }
}
```

Container Query Syntax:

```
/* Establish container */
.card-container {
  container-type: inline-size;
  container-name: card;
}

/* Query the container */
@container card (min-width: 400px) {
  .card {
    display: grid;
    grid-template-columns: 200px 1fr;
  }
}

/* Container query units */
.responsive-text {
  font-size: clamp(1rem, 5cqi, 2rem);
  /* cqi = 1% of container inline size */
}
```

7.4 Example Programs & their Explanation

Example 1: Responsive Page Layout with Media Queries

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Media Query Demo</title>
<style>
  * { box-sizing: border-box; margin: 0; padding: 0; }

  :root {
    --primary: #4361ee;
    --bg: #f5f7fa;
    --text: #1a1a2e;
    --space: 1rem;
  }

  body {
    font-family: system-ui, sans-serif;
    background: var(--bg);
    color: var(--text);
    line-height: 1.6;
  }

  /* Mobile-first base styles */
  .container {
    padding: var(--space);
  }

  .header {
    background: var(--primary);
    color: white;
    padding: 1rem;
    text-align: center;
  }

  .nav {
    display: flex;
    flex-direction: column;
    gap: 0.5rem;
    margin-top: 1rem;
  }

  .nav a {
    color: white;
    text-decoration: none;
    padding: 0.5rem;
    background: rgba(255,255,255,0.1);
    border-radius: 4px;
    text-align: center;
  }

  .grid {
    display: grid;
```

```
  grid-template-columns: 1fr;
  gap: 1rem;
  margin-top: 2rem;
}

.card {
  background: white;
  padding: 1.5rem;
  border-radius: 8px;
  box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}

/* Tablet - 2 columns, horizontal nav */
@media (min-width: 768px) {
  .nav {
    flex-direction: row;
    justify-content: center;
  }

  .grid {
    grid-template-columns: repeat(2, 1fr);
  }

  :root {
    --space: 2rem;
  }
}

/* Desktop - 3 columns, full nav */
@media (min-width: 1024px) {
  .header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    text-align: left;
  }

  .nav {
    margin-top: 0;
  }

  .grid {
    grid-template-columns: repeat(3, 1fr);
  }
}

/* Dark mode preference */
@media (prefers-color-scheme: dark) {
  :root {
    --bg: #1a1a2e;
    --text: #f0f0f0;
  }
  .card {
    background: #2d2d44;
```

```
    }
  }

  /* High resolution displays */
  @media (-webkit-min-device-pixel-ratio: 2), (min-resolution: 192dpi) {
    .card {
      box-shadow: 0 4px 8px rgba(0,0,0,0.15);
    }
  }

  /* Reduced motion */
  @media (prefers-reduced-motion: reduce) {
    * {
      transition: none !important;
      animation: none !important;
    }
  }
</style>
</head>
<body>
  <header class="header">
    <h1>Responsive Demo</h1>
    <nav class="nav">
      <a href="#">Home</a>
      <a href="#">About</a>
      <a href="#">Services</a>
      <a href="#">Contact</a>
    </nav>
  </header>

  <div class="container">
    <h2>Resize the window to see responsive behavior</h2>
    <div class="grid">
      <div class="card">
        <h3>Card 1</h3>
        <p>Adapts to screen size using media queries.</p>
      </div>
      <div class="card">
        <h3>Card 2</h3>
        <p>Uses CSS custom properties for theming.</p>
      </div>
      <div class="card">
        <h3>Card 3</h3>
        <p>Respects user preferences for color and motion.</p>
      </div>
    </div>
  </div>
</body>
</html>
```

Explanation: This example follows a **mobile-first** approach where base styles target mobile devices, then enhance for larger screens. The `min-width` media queries progressively add complexity: at 768px (tablet),

navigation goes horizontal and grid becomes 2 columns; at 1024px (desktop), the header becomes a flex layout and grid has 3 columns. The dark mode media query uses `prefers-color-scheme` to respect system preferences. The high-resolution query enhances shadows on retina displays. The `prefers-reduced-motion` query disables animations for users with motion sensitivity—an important accessibility consideration. CSS custom properties make theme changes elegant and centralized.

Example 2: Container Queries for Component Responsiveness

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Container Query Demo</title>
<style>
  * { box-sizing: border-box; margin: 0; padding: 0; }

  body {
    font-family: system-ui, sans-serif;
    background: #f5f7fa;
    padding: 2rem;
  }

  /* Container that responds to its own size */
  .card-component {
    container-type: inline-size;
    container-name: product-card;
  }

  .card {
    background: white;
    border-radius: 12px;
    overflow: hidden;
    box-shadow: 0 2px 8px rgba(0,0,0,0.1);
    display: flex;
    flex-direction: column;
  }

  .card-image {
    background: linear-gradient(135deg, #667eea, #764ba2);
    aspect-ratio: 16/9;
    display: grid;
    place-items: center;
    color: white;
    font-size: 2rem;
  }

  .card-body {
    padding: 1.5rem;
  }

  .card-title {
    font-size: 1.25rem;
```

```
    margin-bottom: 0.5rem;
  }

  .card-description {
    color: #666;
    font-size: 0.95rem;
  }

  /* Default: stacked layout (narrow container) */
  .meta {
    display: flex;
    flex-direction: column;
    gap: 0.5rem;
    margin-top: 1rem;
    font-size: 0.875rem;
  }

  /* When container is wide enough, switch to horizontal layout */
  @container product-card (min-width: 500px) {
    .card {
      flex-direction: row;
    }

    .card-image {
      flex: 0 0 200px;
      aspect-ratio: auto;
    }

    .meta {
      flex-direction: row;
      gap: 1rem;
    }
  }

  /* When container is very wide, show expanded layout */
  @container product-card (min-width: 700px) {
    .card {
      flex-direction: row;
    }

    .card-image {
      flex: 0 0 300px;
    }

    .card-title {
      font-size: 1.75rem;
    }

    .meta {
      gap: 2rem;
    }
  }

  /* Demo: different sized containers */
```

```
    .demo-grid {
      display: grid;
      grid-template-columns: 1fr;
      gap: 2rem;
      margin-top: 2rem;
    }

    .demo-grid > div {
      border: 2px dashed #ccc;
      padding: 1rem;
      background: rgba(255,255,255,0.5);
    }

    .wide {
      width: 100%;
    }

    .medium {
      max-width: 500px;
    }

    .narrow {
      max-width: 350px;
    }

    .label {
      font-size: 0.75rem;
      color: #666;
      margin-bottom: 0.5rem;
      text-transform: uppercase;
    }
  </style>
  </head>
  <body>
    <h1>Container Queries Demo</h1>
    <p>The same component adapts based on its container's width, not the
  viewport.</p>

    <div class="demo-grid">
      <div>
        <div class="label">Wide Container (responsive grid item)</div>
        <div class="card-component">
          <div class="card">
            <div class="card-image">📦</div>
            <div class="card-body">
              <h3 class="card-title">Product Name</h3>
              <p class="card-description">A description of this amazing product
  with details.</p>
              <div class="meta">
                <span>⭐ 4.8/5</span>
                <span>💬 234 reviews</span>
                <span>📍 Ships worldwide</span>
              </div>
            </div>
```

```
        </div>
      </div>
    </div>

    <div class="medium">
      <div class="label">Medium Container (500px)</div>
      <div class="card-component">
        <div class="card">
          <div class="card-image">📦</div>
          <div class="card-body">
            <h3 class="card-title">Product Name</h3>
            <p class="card-description">A description of this product.</p>
            <div class="meta">
              <span>⭐ 4.8/5</span>
              <span>💬 234 reviews</span>
            </div>
          </div>
        </div>
      </div>
    </div>

    <div class="narrow">
      <div class="label">Narrow Container (350px)</div>
      <div class="card-component">
        <div class="card">
          <div class="card-image">📦</div>
          <div class="card-body">
            <h3 class="card-title">Product Name</h3>
            <p class="card-description">A description of this product.</p>
            <div class="meta">
              <span>⭐ 4.8/5</span>
              <span>💬 234 reviews</span>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
</body>
</html>
```

Explanation: This example demonstrates the power of container queries. The same `.card-component` is used in three different sized containers, and it adapts its layout accordingly—stacked vertically in narrow containers, horizontal in wider ones, and expanded in the widest. The `container-type: inline-size` allows the component to respond to its inline (width) dimension, and the named container `product-card` allows multiple containers in the same document. This is a paradigm shift: components can now be truly responsive based on where they're placed, not just the viewport. This enables component libraries where the same card looks correct in a sidebar, main content area, or modal—without the consumer writing custom CSS.

7.5 Conclusion

Responsive design has evolved from viewport-based media queries to include container queries, enabling truly component-driven responsive design. Media queries remain essential for page-level concerns like navigation patterns and overall layout shifts, while container queries excel at component-level responsiveness. Modern CSS combines both, along with fluid techniques like `clamp()` and viewport units, to create layouts that work across any screen size. The addition of user preference queries (`prefers-color-scheme`, `prefers-reduced-motion`) enables accessible, user-respecting designs. Together, these tools make responsive design more powerful and maintainable than ever.

---

8. Pseudo-classes and Pseudo-elements

8.1 Topic Introduction/Overview

Pseudo-classes and **pseudo-elements** are special CSS selectors that target elements based on state or specific parts of elements that don't exist as separate HTML nodes. They enable sophisticated styling without adding extra markup.

Pseudo-classes (`:`) select elements based on their state, position, or user interaction. Examples include `:hover`, `:focus`, `:nth-child()`, and `:checked`. They style elements in specific conditions.

Pseudo-elements (`::`) create and style specific parts of elements that aren't directly accessible through HTML, like `::before`, `::after`, `::first-line`, and `::first-letter`. They can also generate content.

Together, these selectors dramatically expand what CSS can target and style, enabling interactive designs, decorative effects, and sophisticated content styling with minimal HTML.

8.2 Features, Uses & Advantages

Pseudo-class Categories:

- **User Action**: `:hover`, `:focus`, `:focus-visible`, `:active`
- **Form States**: `:checked`, `:disabled`, `:required`, `:valid`, `:invalid`
- **Structural**: `:first-child`, `:last-child`, `:nth-child()`, `:nth-of-type()`
- **Link States**: `:link`, `:visited`, `:any-link`
- **Logical**: `:not()`, `:is()`, `:where()`, `:has()`
- **Tree-Structural**: `:root`, `:empty`, `:only-child`

Pseudo-elements:

- **`::before` and `::after`**: Generate content before/after an element
- **`::first-line`**: Style the first line of text
- **`::first-letter`**: Style the first letter (drop caps)
- **`::placeholder`**: Style input placeholders
- **`::selection`**: Style highlighted text
- **`::marker`**: Style list bullets/numbers
- **`::backdrop`**: Style fullscreen/dialog backdrops

Advantages:

- No extra HTML needed
- Cleaner, more semantic markup

- Interactive styling
- Sophisticated visual effects
- Content generation

8.3 Syntax & its Explanation

Pseudo-class Syntax:

```
/* Single colon for pseudo-classes */
.button:hover { background: blue; }
.input:focus { border-color: blue; }
.item:first-child { margin-top: 0; }

/* nth-child patterns */
li:nth-child(odd) { background: #f0f0f0; }
li:nth-child(3n) { /* Every 3rd item */ }
li:nth-child(2n+1) { /* Same as odd */ }

/* :not() for negation */
button:not(.disabled) { cursor: pointer; }

/* :is() and :where() for grouping */
:is(h1, h2, h3) { font-family: serif; }
:where(h1, h2, h3) { /* Same but specificity 0,0,0,0 */ }

/* :has() - parent selector */
.card:has(img) { padding: 0; }
form:has(:invalid) { border: 2px solid red; }
```

Pseudo-element Syntax:

```
/* Double colon for pseudo-elements */
.heading::before {
  content: '🎯 ';
  margin-right: 8px;
}

.heading::after {
  content: '';
  display: block;
  width: 50px;
  height: 3px;
  background: blue;
  margin-top: 10px;
}

.paragraph::first-letter {
  font-size: 3em;
  float: left;
}
```

```
input::placeholder {
  color: #999;
  font-style: italic;
}

::selection {
  background: yellow;
  color: black;
}
```

8.4 Example Programs & their Explanation

Example 1: Interactive Form with Pseudo-classes

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Pseudo-classes Demo</title>
<style>
  * { box-sizing: border-box; }

  body {
    font-family: system-ui, sans-serif;
    background: #f5f7fa;
    padding: 2rem;
    max-width: 600px;
    margin: 0 auto;
  }

  .form-group {
    margin-bottom: 1.5rem;
  }

  label {
    display: block;
    margin-bottom: 0.5rem;
    font-weight: 500;
  }

  input, textarea, select {
    width: 100%;
    padding: 0.75rem;
    border: 2px solid #ddd;
    border-radius: 6px;
    font-size: 1rem;
    font-family: inherit;
    transition: border-color 0.2s;
  }
```

```css
/* Focus state - visible only with keyboard */
input:focus {
  outline: none;
  border-color: #4361ee;
}

input:focus-visible {
  outline: 3px solid rgba(67, 97, 238, 0.3);
  outline-offset: 2px;
}

/* Form validation states */
input:required {
  border-left: 3px solid #ff6b6b;
}

input:valid {
  border-color: #43e97b;
}

input:invalid:not(:placeholder-shown) {
  border-color: #ff6b6b;
}

/* Placeholder shown state */
input:placeholder-shown {
  font-style: italic;
}

/* Checkbox and radio custom styling */
.checkbox-group {
  display: flex;
  align-items: center;
  gap: 0.5rem;
}

input[type="checkbox"] {
  width: auto;
  accent-color: #4361ee;
}

input[type="checkbox"]:checked + label {
  color: #4361ee;
  font-weight: 600;
}

/* Submit button states */
button {
  background: #4361ee;
  color: white;
  border: none;
  padding: 0.75rem 2rem;
  border-radius: 6px;
  font-size: 1rem;
```

```
    cursor: pointer;
    transition: all 0.2s;
  }

  button:hover {
    background: #3651d4;
    transform: translateY(-2px);
  }

  button:active {
    transform: translateY(0);
  }

  button:disabled {
    background: #ccc;
    cursor: not-allowed;
    transform: none;
  }

  /* :has() - parent selector */
  .form-group:has(input:invalid:not(:placeholder-shown))::after {
    content: '⚠ Please fill this field correctly';
    color: #ff6b6b;
    font-size: 0.875rem;
    display: block;
    margin-top: 0.25rem;
  }

  /* :is() for multiple selectors */
  :is(h1, h2, h3) {
    color: #1a1a2e;
    margin-bottom: 1rem;
  }
</style>
</head>
<body>
  <h1>Interactive Form</h1>

  <form>
    <div class="form-group">
      <label for="name">Name (required)</label>
      <input type="text" id="name" required placeholder="Enter your name">
    </div>

    <div class="form-group">
      <label for="email">Email (required)</label>
      <input type="email" id="email" required placeholder="you@example.com">
    </div>

    <div class="form-group">
      <label for="phone">Phone</label>
      <input type="tel" id="phone" placeholder="Optional">
    </div>
```

```
    <div class="form-group checkbox-group">
      <input type="checkbox" id="subscribe">
      <label for="subscribe">Subscribe to newsletter</label>
    </div>

    <button type="submit">Submit Form</button>
  </form>
</body>
</html>
```

Explanation: This example showcases numerous pseudo-classes for form styling. `:required` adds a red left border, `:valid`/`invalid` change border colors based on validation, `:placeholder-shown` detects if the placeholder is visible. The `:checked` pseudo-class with adjacent sibling selector (`+`) highlights the label when a checkbox is checked. `:focus-visible` adds an outline only for keyboard navigation (better than `:focus` for accessibility). The `:has()` pseudo-class enables the powerful pattern of styling a parent based on its children's state—showing an error message when an input is invalid. `:is()` groups selectors without increasing specificity.

Example 2: Decorative Effects with Pseudo-elements

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Pseudo-elements Demo</title>
<style>
  * { box-sizing: border-box; }

  body {
    font-family: system-ui, sans-serif;
    background: #f5f7fa;
    padding: 2rem;
    max-width: 800px;
    margin: 0 auto;
    line-height: 1.6;
  }

  /* Decorative heading with ::before and ::after */
  .fancy-heading {
    text-align: center;
    font-size: 2.5rem;
    margin: 2rem 0;
    display: flex;
    align-items: center;
    justify-content: center;
    gap: 1rem;
  }

  .fancy-heading::before,
  .fancy-heading::after {
    content: '';
```

```
  flex: 1;
  height: 2px;
  background: linear-gradient(to right, transparent, #4361ee, transparent);
  max-width: 200px;
}

/* Tooltip with pseudo-elements */
.tooltip {
  position: relative;
  display: inline-block;
  border-bottom: 2px dotted #4361ee;
  cursor: help;
}

.tooltip::before {
  content: attr(data-tooltip);
  position: absolute;
  bottom: 100%;
  left: 50%;
  transform: translateX(-50%) translateY(-10px);
  background: #1a1a2e;
  color: white;
  padding: 0.5rem 1rem;
  border-radius: 6px;
  font-size: 0.875rem;
  white-space: nowrap;
  opacity: 0;
  pointer-events: none;
  transition: all 0.2s;
}

.tooltip::after {
  content: '';
  position: absolute;
  bottom: 100%;
  left: 50%;
  transform: translateX(-50%);
  border: 6px solid transparent;
  border-top-color: #1a1a2e;
  opacity: 0;
  transition: opacity 0.2s;
}

.tooltip:hover::before,
.tooltip:hover::after {
  opacity: 1;
}

.tooltip:hover::before {
  transform: translateX(-50%) translateY(-5px);
}

/* Custom list markers */
.custom-list {
```

```
  list-style: none;
  padding: 0;
}

.custom-list li {
  position: relative;
  padding-left: 2.5rem;
  margin-bottom: 0.75rem;
}

.custom-list li::marker {
  color: #4361ee;
  font-weight: bold;
}

.custom-list li::before {
  content: '✓';
  position: absolute;
  left: 0;
  top: 0;
  width: 1.75rem;
  height: 1.75rem;
  background: #4361ee;
  color: white;
  border-radius: 50%;
  display: grid;
  place-items: center;
  font-weight: bold;
}

/* Article with first-letter drop cap */
.article {
  background: white;
  padding: 2rem;
  border-radius: 8px;
  box-shadow: 0 2px 4px rgba(0,0,0,0.1);
  margin: 2rem 0;
}

.article p {
  margin-bottom: 1rem;
}

.article p::first-letter {
  font-size: 4em;
  float: left;
  line-height: 0.85;
  margin: 0.1em 0.1em 0 0;
  color: #4361ee;
  font-weight: bold;
}

/* Quote with decorative quotes */
blockquote {
```

```
    background: white;
    border-left: 4px solid #4361ee;
    padding: 1.5rem 2rem;
    margin: 2rem 0;
    font-style: italic;
    position: relative;
  }

  blockquote::before {
    content: '"';
    position: absolute;
    top: -20px;
    left: 10px;
    font-size: 5rem;
    color: #4361ee;
    opacity: 0.2;
    font-family: Georgia, serif;
  }

  /* Text selection styling */
  ::selection {
    background: #4361ee;
    color: white;
  }

  /* Required field indicator */
  label.required::after {
    content: ' *';
    color: #ff6b6b;
  }
</style>
</head>
<body>
  <h1 class="fancy-heading">Pseudo-elements in Action</h1>

  <p>Hover over this <span class="tooltip" data-tooltip="I'm a
tooltip!">underlined text</span> to see a tooltip.</p>

  <h2>Custom List with ::before</h2>
  <ul class="custom-list">
    <li>First item with custom checkmark</li>
    <li>Second item with custom checkmark</li>
    <li>Third item with custom checkmark</li>
  </ul>

  <div class="article">
    <p>This article uses the ::first-letter pseudo-element to create a drop cap
effect.
       The first letter is enlarged, floated to the left, and colored
differently
       to create visual interest and traditional newspaper-style typography.
</p>
    <p>The ::first-line pseudo-element could style just the first line
differently,
```

```
        perhaps in small caps or with a different color for emphasis.</p>
    </div>

    <blockquote>
      Pseudo-elements enable creative designs without additional HTML, keeping
markup clean
      while adding visual richness through CSS alone.
    </blockquote>
  </body>
  </html>
```

Explanation: This example demonstrates creative uses of pseudo-elements. The `::before` and `::after` on the heading create decorative lines using gradients. The tooltip uses `content: attr(data-tooltip)` to display text from an HTML attribute, with both `::before` (the tooltip box) and `::after` (the arrow). The custom list uses `::before` for checkmarks in circles. The article's `::first-letter` creates a drop cap. The blockquote uses a giant `::before` quote mark for decoration. `::selection` styles the highlighted text when users select it. The `::after` on required labels adds an asterisk. Note how all these effects require zero additional HTML—pure CSS power.

8.5 Conclusion

Pseudo-classes and pseudo-elements are among CSS's most powerful features, enabling sophisticated styling and interaction without HTML bloat. Pseudo-classes respond to state and structure, while pseudo-elements access and create content within elements. The introduction of `:has()` (the "parent selector") and the logical `:is()`/`:where()` has made CSS even more capable. Combined with `::before`/`::after` for content generation, these features enable complex visual effects, interactive forms, and decorative designs with minimal markup. As CSS continues to evolve, these selectors remain fundamental tools for every developer.

---

9. Transitions, Animations, and Scroll-driven Animations

9.1 Topic Introduction/Overview

CSS provides three main techniques for creating motion on the web: **transitions**, **animations**, and the newer **scroll-driven animations**. These allow developers to create engaging, interactive experiences without JavaScript.

Transitions enable smooth changes between property values when an element's state changes (like on hover or focus). They interpolate values over a duration with optional easing.

CSS Animations (`@keyframes`) allow complex, multi-step animations that can run automatically, loop, or play on demand. They can animate multiple properties simultaneously with precise timing control.

Scroll-driven Animations (newer) tie animation progress to scroll position rather than time. This enables effects like parallax scrolling, scroll-triggered reveals, and progress indicators that respond to how far the user has scrolled.

Together, these features enable modern, performant web animations that enhance user experience when used thoughtfully.

9.2 Features, Uses & Advantages

Transition Features:

- **Properties**: `transition-property`, `transition-duration`, `transition-timing-function`, `transition-delay`
- **Timing Functions**: `ease`, `linear`, `ease-in`, `ease-out`, `ease-in-out`, `cubic-bezier()`, `steps()`
- **Shorthand**: `transition: property duration timing-function delay`

Animation Features:

- `@keyframes`: Define animation steps with percentages
- **Properties**: `animation-name`, `animation-duration`, `animation-timing-function`, `animation-delay`, `animation-iteration-count`, `animation-direction`, `animation-fill-mode`, `animation-play-state`
- `animation-composition`: Combine multiple animations (replace, add, accumulate)
- `@scope`: Limit animation scope

Scroll-driven Animation Features:

- `animation-timeline`: `scroll()`, `view()`, or custom named timeline
- `animation-range`: Define when animation starts/ends within timeline
- **Scroll Functions**: `scroll()` for element/container scroll, `view()` for element in viewport

Advantages:

- GPU-accelerated performance
- No JavaScript needed
- Better user engagement
- Smoother UX
- Reduced motion can be respected via `prefers-reduced-motion`

9.3 Syntax & its Explanation

Transition Syntax:

```
.button {
  background: blue;
  transition: background 0.3s ease, transform 0.2s ease-out;
}

.button:hover {
  background: darkblue;
  transform: scale(1.05);
}

/* Individual properties */
.box {
  transition-property: all;
  transition-duration: 300ms;
  transition-timing-function: cubic-bezier(0.4, 0, 0.2, 1);
```

```
  transition-delay: 0s;
}
```

Animation Syntax:

```
@keyframes slideIn {
  from {
    transform: translateX(-100%);
    opacity: 0;
  }
  to {
    transform: translateX(0);
    opacity: 1;
  }
}

@keyframes bounce {
  0%, 100% { transform: translateY(0); }
  50% { transform: translateY(-20px); }
}

.element {
  animation: slideIn 0.5s ease-out forwards;
  /* name duration timing-function delay iteration-count direction fill-mode
play-state */
}

.loop {
  animation: bounce 2s ease-in-out infinite;
}
```

Scroll-driven Animation Syntax:

```
@keyframes fadeIn {
  to { opacity: 1; transform: translateY(0); }
}

.fade-in-element {
  opacity: 0;
  transform: translateY(50px);
  animation: fadeIn linear both;
  animation-timeline: view();
  animation-range: entry 0% cover 30%;
}
```

9.4 Example Programs & their Explanation

Example 1: Transitions and Keyframe Animations

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Animations Demo</title>
<style>
  * { box-sizing: border-box; }

  body {
    font-family: system-ui, sans-serif;
    background: #f5f7fa;
    padding: 2rem;
    max-width: 1000px;
    margin: 0 auto;
  }

  h2 {
    margin: 2rem 0 1rem;
    color: #1a1a2e;
  }

  /* Transition Examples */
  .button-group {
    display: flex;
    gap: 1rem;
    flex-wrap: wrap;
    margin-bottom: 2rem;
  }

  .btn {
    padding: 0.75rem 1.5rem;
    background: #4361ee;
    color: white;
    border: none;
    border-radius: 6px;
    cursor: pointer;
    font-size: 1rem;
  }

  /* Smooth transition */
  .btn-smooth {
    transition: all 0.3s cubic-bezier(0.4, 0, 0.2, 1);
  }

  .btn-smooth:hover {
    background: #3651d4;
    transform: translateY(-3px);
    box-shadow: 0 10px 20px rgba(67, 97, 238, 0.3);
  }

  /* Bounce transition */
  .btn-bounce {
```

```
  transition: transform 0.2s cubic-bezier(0.68, -0.55, 0.265, 1.55);
}

.btn-bounce:hover {
  transform: scale(1.1);
}

.btn-bounce:active {
  transform: scale(0.95);
}

/* Loading Spinner - Keyframe Animation */
.spinner {
  width: 40px;
  height: 40px;
  border: 4px solid rgba(67, 97, 238, 0.2);
  border-top-color: #4361ee;
  border-radius: 50%;
  animation: spin 1s linear infinite;
}

@keyframes spin {
  to { transform: rotate(360deg); }
}

/* Pulse Animation */
.pulse-dot {
  width: 20px;
  height: 20px;
  background: #ff6b6b;
  border-radius: 50%;
  animation: pulse 1.5s ease-in-out infinite;
}

@keyframes pulse {
  0%, 100% {
    transform: scale(1);
    opacity: 1;
  }
  50% {
    transform: scale(1.5);
    opacity: 0.5;
  }
}

/* Slide-in Animation */
.slide-in-box {
  width: 100px;
  height: 100px;
  background: linear-gradient(135deg, #667eea, #764ba2);
  border-radius: 8px;
  animation: slideIn 0.6s ease-out;
}
```

```
@keyframes slideIn {
  from {
    transform: translateX(-200px) rotate(-45deg);
    opacity: 0;
  }
  to {
    transform: translateX(0) rotate(0);
    opacity: 1;
  }
}

/* Complex Multi-step Animation */
.orbit {
  width: 200px;
  height: 200px;
  position: relative;
  margin: 2rem auto;
}

.orbit-dot {
  position: absolute;
  top: 50%;
  left: 50%;
  width: 30px;
  height: 30px;
  background: #4361ee;
  border-radius: 50%;
  margin: -15px;
  animation: orbit 3s linear infinite;
  transform-origin: center;
}

@keyframes orbit {
  from {
    transform: rotate(0deg) translateX(80px) rotate(0deg);
  }
  to {
    transform: rotate(360deg) translateX(80px) rotate(-360deg);
  }
}

/* Animation Demo Container */
.demo-box {
  background: white;
  padding: 2rem;
  border-radius: 8px;
  margin: 1rem 0;
  display: flex;
  align-items: center;
  gap: 2rem;
  min-height: 150px;
}

/* Respect reduced motion */
```

```
    @media (prefers-reduced-motion: reduce) {
      * {
        animation-duration: 0.01ms !important;
        animation-iteration-count: 1 !important;
        transition-duration: 0.01ms !important;
      }
    }
  </style>
  </head>
  <body>
    <h1>CSS Animations & Transitions</h1>

    <h2>Transitions on Hover</h2>
    <div class="button-group">
      <button class="btn btn-smooth">Smooth Button</button>
      <button class="btn btn-bounce">Bouncy Button</button>
    </div>

    <h2>Loading Spinner</h2>
    <div class="demo-box">
      <div class="spinner"></div>
    </div>

    <h2>Pulse Effect</h2>
    <div class="demo-box">
      <div class="pulse-dot"></div>
    </div>

    <h2>Slide-in Animation</h2>
    <div class="demo-box">
      <div class="slide-in-box"></div>
    </div>

    <h2>Orbit Animation</h2>
    <div class="demo-box">
      <div class="orbit">
        <div class="orbit-dot"></div>
      </div>
    </div>
  </body>
  </html>
```

Explanation: This example demonstrates various animation techniques. The **buttons** use transitions with different cubic-bezier timing functions—the smooth one uses Material Design's standard easing, while the bouncy one uses a back-easing curve that overshoots before settling. The **spinner** uses a simple keyframe animation that rotates infinitely. The **pulse** uses 0%, 50%, and 100% keyframes for a heartbeat effect. The **slide-in** combines translation and rotation. The **orbit** uses a clever two-rotation transform: the first `rotate(0deg) translateX(80px)` moves the dot to its position, and the second `rotate(0deg)` (or `-360deg` at end) keeps it upright as it orbits. The reduced motion media query respects user preferences.

Example 2: Scroll-driven Animations

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Scroll Animations</title>
<style>
  * { box-sizing: border-box; }

  body {
    font-family: system-ui, sans-serif;
    margin: 0;
    background: #f5f7fa;
  }

  /* Long page for scrolling */
  .hero {
    height: 100vh;
    display: grid;
    place-items: center;
    background: linear-gradient(135deg, #667eea, #764ba2);
    color: white;
    text-align: center;
    position: relative;
  }

  .hero h1 {
    font-size: 4rem;
    margin: 0;
  }

  .hero p {
    font-size: 1.5rem;
    margin-top: 1rem;
  }

  /* Parallax effect */
  .parallax-bg {
    position: absolute;
    inset: 0;
    background: radial-gradient(circle, rgba(255,255,255,0.1) 1px, transparent
1px);
    background-size: 30px 30px;
    animation: parallax linear;
    animation-timeline: scroll(root);
    animation-range: 0 100vh;
  }

  @keyframes parallax {
    to {
      transform: translateY(-50px);
      background-size: 50px 50px;
    }
```

```
}

/* Progress bar */
.progress-bar {
  position: fixed;
  top: 0;
  left: 0;
  height: 4px;
  background: #4361ee;
  transform-origin: left;
  animation: progress linear;
  animation-timeline: scroll(root);
  animation-range: 0 100vh;
  z-index: 1000;
}

@keyframes progress {
  to { transform: scaleX(1); }
}

/* Content sections */
.content {
  max-width: 800px;
  margin: 0 auto;
  padding: 4rem 2rem;
}

.content h2 {
  font-size: 2.5rem;
  color: #1a1a2e;
  margin-bottom: 1rem;
}

/* Scroll-triggered reveal */
.reveal {
  opacity: 0;
  transform: translateY(50px);
  animation: reveal linear forwards;
  animation-timeline: view();
  animation-range: entry 0% cover 40%;
}

@keyframes reveal {
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

/* Scale on scroll */
.scale-on-scroll {
  width: 100%;
  aspect-ratio: 16/9;
  background: linear-gradient(45deg, #f093fb, #f5576c);
```

```
      border-radius: 12px;
      transform: scale(0.8);
      animation: scaleUp linear;
      animation-timeline: view();
      animation-range: entry 20% cover 50%;
    }

    @keyframes scaleUp {
      to { transform: scale(1); }
    }

    /* Card deck effect */
    .card-stack {
      perspective: 1000px;
      height: 300px;
      position: relative;
    }

    .stack-card {
      position: absolute;
      inset: 0;
      background: white;
      border-radius: 12px;
      box-shadow: 0 10px 30px rgba(0,0,0,0.1);
      animation: cardReveal linear both;
      animation-timeline: view();
      animation-range: entry 10% cover 50%;
    }

    .stack-card:nth-child(1) { animation-delay: 0s; }
    .stack-card:nth-child(2) { animation-delay: 0.2s; }
    .stack-card:nth-child(3) { animation-delay: 0.4s; }

    @keyframes cardReveal {
      from {
        transform: translateY(50px) rotateX(-20deg);
        opacity: 0;
      }
      to {
        transform: translateY(0) rotateX(0);
        opacity: 1;
      }
    }

    /* Footer spacer */
    .footer-spacer {
      height: 50vh;
      background: linear-gradient(180deg, #f5f7fa, #1a1a2e);
      color: white;
      display: grid;
      place-items: center;
    }
  </style>
  </head>
```

```
<body>
  <div class="progress-bar"></div>

  <section class="hero">
    <div class="parallax-bg"></div>
    <div>
      <h1>Scroll Down</h1>
      <p>Watch the animations as you scroll</p>
    </div>
  </section>

  <div class="content">
    <h2 class="reveal">Scroll-triggered Animations</h2>
    <p class="reveal">These elements animate as they enter the viewport.
       The animation is tied to scroll position using the view() timeline.</p>
  </div>

  <div class="content">
    <h2 class="reveal">Scale on Scroll</h2>
    <div class="scale-on-scroll reveal"></div>
    <p class="reveal">This element scales up as it scrolls into view.</p>
  </div>

  <div class="content">
    <h2 class="reveal">3D Card Reveal</h2>
    <div class="card-stack reveal">
      <div class="stack-card" style="background: linear-gradient(135deg,
#667eea, #764ba2);"></div>
      <div class="stack-card" style="background: linear-gradient(135deg,
#f093fb, #f5576c); transform: translateY(20px);"></div>
      <div class="stack-card" style="background: linear-gradient(135deg,
#4facfe, #00f2fe); transform: translateY(40px);"></div>
    </div>
  </div>

  <div class="content">
    <h2 class="reveal">Keep Scrolling</h2>
    <p class="reveal">The progress bar at the top shows your scroll progress.
       Scroll-driven animations tie animation progress to scroll position
rather than time.</p>
  </div>

  <div class="footer-spacer">
    <h2>End of Page</h2>
  </div>
</body>
</html>
```

Explanation: This example showcases the modern **scroll-driven animations** API. The `progress-bar` uses `animation-timeline: scroll(root)` to track the root scrollbar—scaling from 0 to 1 as you scroll. The **parallax** background moves up slower than scroll. The **reveal** animations use `animation-timeline: view()` and `animation-range: entry 0% cover 40%` to animate elements as they enter the viewport—

opacity and transform interpolate based on how much of the element is visible. The **scale-on-scroll** uses similar view-timeline behavior. The 3D **card stack** demonstrates how view timelines work with transforms, creating a reveal effect. These animations are more performant than JavaScript scroll handlers because they're handled by the browser's compositor.

9.5 Conclusion

CSS animations have evolved from simple transitions to complex, scroll-driven experiences. **Transitions** provide state-change animations, **@keyframes** enable complex multi-step animations, and **scroll-driven animations** tie animation progress to scroll position. The key to good animation is restraint—use them purposefully to enhance UX, not distract. Always respect `prefers-reduced-motion` for accessibility. Modern CSS animations are GPU-accelerated and performant, making them ideal for production use. As the web platform evolves, expect even more powerful animation primitives that reduce JavaScript dependency.

---

10. Modern CSS & Variables, calc(), clamp(), Nesting, @layer

10.1 Topic Introduction/Overview

Modern CSS encompasses the powerful features introduced in recent specifications that have transformed how we write stylesheets. These features make CSS more maintainable, dynamic, and powerful than ever before.

CSS Custom Properties (Variables) allow you to store and reuse values throughout your stylesheet, enabling theming, dynamic updates, and more maintainable code.

Math Functions like `calc()`, `min()`, `max()`, and `clamp()` enable dynamic values that respond to context—essential for responsive design without media queries.

CSS Nesting lets you nest selectors within selectors, mirroring the structure of your HTML and reducing repetition.

`@layer` provides explicit control over the cascade, allowing you to define clear priorities for different parts of your stylesheet.

Together, these modern features represent CSS's evolution into a more powerful, programmatic styling language while maintaining its declarative nature.

10.2 Features, Uses & Advantages

Custom Properties Features:

- **Definition**: `--variable-name: value;`
- **Usage**: `var(--variable-name, fallback)`
- **Inheritance**: Custom properties inherit by default
- **Dynamic Updates**: Change with JavaScript or other CSS
- **Type Checking**: Can be registered with `@property` for type safety

Math Functions:

- `calc()`: Mathematical operations with mixed units

- `min()` / `max()`: Choose smallest/largest value from list
- `clamp()`: Constrain value between min and max
- `round()`: Round to nearest multiple
- `mod()`, `rem()`, `abs()`, `sign()`: Other math functions
- `sin()`, `cos()`, `tan()`: Trigonometric functions

Nesting Features:

- **Selector Nesting**: Nest any selector within another
- `&` **Reference**: Parent selector reference
- **Media Query Nesting**: Nest `@media` inside selectors
- **Conditional Nesting**: Nest `@supports`, `@container`, etc.

`@layer` **Features:**

- **Layer Declaration**: `@layer layerName { ... }`
- **Layer Order**: First declared = lowest priority
- **Anonymous Layers**: `@layer { ... }`
- **Nested Layers**: Layers within layers
- **Important Layers**: `@layer !layerName`

Advantages:

- More maintainable code
- Dynamic theming
- Less repetition
- Better organization
- Type-safe values (with `@property`)
- Explicit cascade control

10.3 Syntax & its Explanation

Custom Properties Syntax:

```
:root {
  --primary: #4361ee;
  --spacing-unit: 8px;
  --max-width: 1200px;
}

.button {
  background: var(--primary);
  padding: calc(var(--spacing-unit) * 2);
  max-width: var(--max-width);
}

/* Type registration */
@property --gradient-angle {
  syntax: '<angle>';
  initial-value: 0deg;
```

```
  inherits: false;
}
```

Math Function Syntax:

```
/* calc - mixed units */
.box {
  width: calc(100% - 40px);
  margin: calc(var(--gap) * 2);
}

/* clamp - min, preferred, max */
.heading {
  font-size: clamp(1.5rem, 4vw, 3rem);
  /* Min 1.5rem, preferred 4vw, max 3rem */
}

/* min and max */
.box {
  width: min(100%, 800px);
  /* Smaller of 100% or 800px */
  padding: max(20px, 5vw);
  /* Larger of 20px or 5vw */
}
```

Nesting Syntax:

```
.card {
  padding: 1rem;
  background: white;

  & .title {
    font-size: 1.5rem;
  }

  &:hover {
    transform: translateY(-5px);
  }

  & > .content {
    margin-top: 1rem;
  }

  @media (min-width: 768px) {
    padding: 2rem;
  }
}
```

@layer Syntax:

```
/* Define layer order */
@layer reset, base, components, utilities;

@layer reset {
  * { margin: 0; padding: 0; }
}

@layer base {
  body { font-family: sans-serif; }
}

@layer components {
  .button { padding: 0.5rem 1rem; }
}

/* !important layers reverse priority */
@layer !utilities {
  .text-center { text-align: center !important; }
}
```

10.4 Example Programs & their Explanation

Example 1: Theming with Custom Properties and Math Functions

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Modern CSS Demo</title>
<style>
  /* Custom Properties for theming */
  :root {
    /* Colors */
    --color-primary: #4361ee;
    --color-bg: #ffffff;
    --color-text: #1a1a2e;
    --color-border: #e0e0e0;

    /* Spacing scale */
    --space-xs: 0.25rem;
    --space-sm: 0.5rem;
    --space-md: 1rem;
    --space-lg: 2rem;
    --space-xl: 4rem;

    /* Typography */
    --font-base: system-ui, -apple-system, sans-serif;
    --font-size-base: clamp(1rem, 0.9rem + 0.5vw, 1.125rem);
```

```
  --line-height: 1.6;
}

/* Dark theme override */
[data-theme="dark"] {
  --color-bg: #1a1a2e;
  --color-text: #f0f0f0;
  --color-border: #2d2d44;
}

/* Type registration for smooth animation */
@property --gradient-angle {
  syntax: '<angle>';
  initial-value: 0deg;
  inherits: false;
}

* { box-sizing: border-box; margin: 0; padding: 0; }

body {
  font-family: var(--font-base);
  font-size: var(--font-size-base);
  line-height: var(--line-height);
  background: var(--color-bg);
  color: var(--color-text);
  transition: background 0.3s, color 0.3s;
}

.container {
  max-width: min(90%, 1200px);
  margin: 0 auto;
  padding: var(--space-lg);
}

/* Responsive typography with clamp() */
h1 {
  font-size: clamp(2rem, 1.5rem + 3vw, 4rem);
  margin-bottom: var(--space-md);
  line-height: 1.1;
}

h2 {
  font-size: clamp(1.5rem, 1.2rem + 1.5vw, 2.5rem);
  margin-bottom: var(--space-md);
}

/* Layout using calc() */
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(min(100%, 300px), 1fr));
  gap: calc(var(--space-md) * 1.5);
  margin: var(--space-xl) 0;
}
```

```
  .card {
    background: var(--color-bg);
    border: 1px solid var(--color-border);
    border-radius: 12px;
    padding: var(--space-lg);
    transition: transform 0.2s;
  }

  .card:hover {
    transform: translateY(calc(var(--space-sm) * -1));
  }

  /* Animated gradient with custom property */
  .gradient-text {
    --gradient-angle: 0deg;
    background: linear-gradient(
      var(--gradient-angle),
      var(--color-primary),
      #f093fb,
      #4facfe,
      var(--color-primary)
    );
    background-size: 200% 200%;
    -webkit-background-clip: text;
    background-clip: text;
    -webkit-text-fill-color: transparent;
    color: transparent;
    animation: rotate-gradient 5s linear infinite;
  }

  @keyframes rotate-gradient {
    to { --gradient-angle: 360deg; }
  }

  /* Theme toggle button */
  .theme-toggle {
    position: fixed;
    top: var(--space-md);
    right: var(--space-md);
    padding: var(--space-sm) var(--space-md);
    background: var(--color-primary);
    color: white;
    border: none;
    border-radius: 6px;
    cursor: pointer;
    font-size: 1rem;
  }

  /* Button with responsive padding */
  .button {
    display: inline-block;
    padding: clamp(0.5rem, 0.4rem + 0.5vw, 1rem) clamp(1rem, 0.8rem + 1vw,
2rem);
    background: var(--color-primary);
```

```
      color: white;
      border: none;
      border-radius: 6px;
      cursor: pointer;
      font-size: 1rem;
      margin-top: var(--space-md);
    }

    /* Fluid spacing */
    .spacer {
      height: clamp(2rem, 1rem + 5vw, 6rem);
    }
  </style>
  </head>
  <body>
    <button class="theme-toggle" onclick="document.documentElement.dataset.theme
  = document.documentElement.dataset.theme === 'dark' ? '' : 'dark'">
      Toggle Theme
    </button>

    <div class="container">
      <h1 class="gradient-text">Modern CSS Features</h1>
      <p>This page uses CSS custom properties, math functions, and type
  registration
         for a fully themable, responsive experience.</p>

      <div class="spacer"></div>

      <h2>Responsive Grid</h2>
      <div class="grid">
        <div class="card">
          <h3>Custom Properties</h3>
          <p>Reusable values that can be themed dynamically.</p>
        </div>
        <div class="card">
          <h3>Math Functions</h3>
          <p>calc(), clamp(), min(), max() for fluid values.</p>
        </div>
        <div class="card">
          <h3>Type Registration</h3>
          <p>@property enables animatable custom properties.</p>
        </div>
      </div>

      <button class="button">Responsive Button</button>
    </div>
  </body>
  </html>
```

Explanation: This example showcases modern CSS features. **Custom properties** in `:root` define a design system—colors, spacing, typography. The `[data-theme="dark"]` selector overrides these for dark mode, and toggling the `data-theme` attribute switches themes instantly. **`clamp()`** creates fluid typography and padding

that scales smoothly between screen sizes. `calc()` combines units (e.g., `calc(var(--space-sm) * -1)` for hover offset). `@property` registers the `--gradient-angle` as an angle type, enabling the browser to interpolate it smoothly in the keyframe animation—something not possible with regular custom properties. `min()` in the grid uses `min(100%, 300px)` to ensure columns never exceed their container.

Example 2: Nesting and @layer Organization

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Nesting & Layers</title>
<style>
  /* Define layer order - earlier = lower priority */
  @layer reset, base, components, utilities;

  /* Reset layer - lowest priority */
  @layer reset {
    *, *::before, *::after {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }
  }

  /* Base layer */
  @layer base {
    body {
      font-family: system-ui, sans-serif;
      line-height: 1.6;
      background: #f5f7fa;
      color: #1a1a2e;
    }

    a {
      color: #4361ee;
      text-decoration: none;
    }
  }

  /* CSS Nesting - modern syntax */
  @layer components {
    .card {
      background: white;
      border-radius: 12px;
      padding: 1.5rem;
      box-shadow: 0 2px 8px rgba(0,0,0,0.1);
      max-width: 400px;
      margin: 2rem auto;

      /* Nested selector with & */
      & h2 {
```

```
    color: #1a1a2e;
    margin-bottom: 0.5rem;
  }

  & p {
    color: #666;
    margin-bottom: 1rem;
  }

  /* Pseudo-class nesting */
  &:hover {
    transform: translateY(-4px);
    box-shadow: 0 10px 20px rgba(0,0,0,0.15);
  }

  /* Compound selector */
  &.featured {
    border: 2px solid #4361ee;

    & h2 {
      color: #4361ee;
    }
  }

  /* Descendant selector */
  & .actions {
    display: flex;
    gap: 0.5rem;
    margin-top: 1rem;
  }

  /* Nested media query */
  @media (min-width: 768px) {
    padding: 2rem;
  }
}

.button {
  padding: 0.5rem 1rem;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  font-size: 0.875rem;

  &.primary {
    background: #4361ee;
    color: white;
  }

  &.secondary {
    background: transparent;
    color: #4361ee;
    border: 1px solid #4361ee;
  }
```

```
      &:hover {
        opacity: 0.9;
      }
    }
  }

  /* Utility layer - highest priority */
  @layer utilities {
    .text-center { text-align: center; }
    .mt-4 { margin-top: 1rem; }
    .mb-4 { margin-bottom: 1rem; }

    /* !important utilities layer */
    @layer !overrides {
      .no-shadow { box-shadow: none !important; }
    }
  }

  /* Unlayered styles - highest priority of all */
  .emergency-override {
    background: red !important;
    color: white !important;
  }
</style>
</head>
<body>
  <div class="card text-center">
    <h2>Nested Card Component</h2>
    <p>This uses CSS nesting to keep related styles together.
       The structure mirrors the HTML, making it easier to read.</p>
    <div class="actions">
      <button class="button primary">Primary</button>
      <button class="button secondary">Secondary</button>
    </div>
  </div>

  <div class="card featured text-center">
    <h2>Featured Card</h2>
    <p>The &amp;.featured compound selector is nested, showing how nesting
       can express relationships clearly.</p>
    <div class="actions">
      <button class="button primary">Learn More</button>
    </div>
  </div>
</body>
</html>
```

Explanation: This example demonstrates **CSS Nesting** and `@layer`. The nesting eliminates repetition—you write `.card` once and nest `& h2`, `&:hover`, `&.featured`, `& .actions` inside it. The `&` represents the parent selector, making relationships clear. The media query is nested inside `.card`, applying only to that component. The `@layer` **declarations** at the top (`@layer reset, base, components, utilities`) establish a clear

priority order—utilities always win over components, which always win over base styles. The `!overrides` sub-layer uses `!` to invert priority. **Unlayered styles** (outside any `@layer`) have the highest priority of all, making the `.emergency-override` class beat everything. This explicit cascade control is invaluable in large codebases with multiple authors.

10.5 Conclusion

Modern CSS has transformed from a simple styling language into a powerful, programmatic system. **Custom properties** enable theming and dynamic values. **Math functions** like `clamp()` and `calc()` make responsive design fluid without breakpoints. **Nesting** improves readability and reduces repetition. **`@layer`** provides explicit control over the cascade, solving specificity wars in large codebases. Together, these features enable design systems, component libraries, and maintainable stylesheets at scale. As CSS continues to evolve with features like `@property` for type safety and `color-mix()` for color manipulation, the language is becoming more capable than ever—all while maintaining its declarative, accessible nature.

Final Summary

This guide has covered the essential CSS topics from fundamentals to modern features. Here's a quick recap of what you've learned:

1. **Fundamentals & Cascade**: Selectors, specificity, and how the cascade resolves conflicts
2. **Box Model & Typography**: The foundation of layout and text presentation
3. **Colors & Gradients**: Modern color formats and gradient techniques
4. **Flexbox**: One-dimensional layout for components
5. **Grid**: Two-dimensional layout for pages and complex interfaces
6. **Positioning**: Layered elements and stacking contexts
7. **Responsive Design**: Media queries and container queries for all devices
8. **Pseudo-classes/elements**: Powerful selectors for state and decoration
9. **Animations**: Transitions, keyframes, and scroll-driven motion
10. **Modern CSS**: Variables, math, nesting, and layers for maintainable code

These topics build on each other—mastering the fundamentals enables you to use advanced features effectively. Modern CSS is more powerful than ever, and these tools enable you to create responsive, accessible, maintainable designs with less code and better performance.